



**UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH**



**telecos
BCN**



DESIGN AND DEVELOPMENT OF A VISUAL ANALYTICS APPLICATION FOR MULTI-STAGE BICYCLE RACES

ADRIAN QUIRANTE GONZÁLEZ

Thesis supervisor

PERE PAU VÁZQUEZ ALCOCER (Department of Computer Science)

Degree

Bachelor's Degree in Data Science and Engineering

Bachelor's thesis

Facultat d'Informàtica de Barcelona (FIB)

Escola Tècnica Superior d'Enginyeria de Telecomunicació de Barcelona (ETSETB)

Facultat de Matemàtiques i Estadística (FME)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

This thesis is the result of months of work that would not have been possible without the support of many people. I would like to take this opportunity to thank all of them.

First and foremost, I would like to thank Pere-Pau Vázquez and José Díaz for their guidance and support throughout the course of the project. Beyond their expertise in data visualization and multi-stage bicycle races, they managed to transmit their genuine passion for the sport, which became a constant source of motivation throughout the whole process. Thank you for your patience, your availability, and your encouragement to keep going.

I also want to thank the UPC, and in particular the GCED professors and directors, for the quality of the degree and its subjects, which provided me with the knowledge and confidence to take on and develop a project of this scope.

Finally, I would like to thank my family and classmates, whose support throughout the thesis made the whole process considerably more enjoyable.

Abstract

Professional cycling generates large amounts of high-frequency telemetry data during multi-stage races, yet existing tools offer little beyond final results and basic statistics, leaving in-race dynamics largely inaccessible. This project proposes a visual analytics web application designed to exploit per-rider telemetry data — such as speed, on-road position, group membership, and time gaps — sampled every 10 seconds, to enable deep exploration and analysis of individual race stages. Using Stage 7 of the 2021 Tour de France as a baseline, an interactive interface was developed with React and FastAPI, combining Plotly and Apache ECharts to deliver rich, interactive visualizations, allowing users to examine the race at any selected moment or interval and extract overall, rider, and team performance analysis across the full stage. Evaluated against a set of research questions spanning three analytical levels, the application aims to provide from cycling fans to analysts with a powerful tool to understand at a high level of detail the development of each stage and race.

Keywords

Visual analytics, Multi-stage bicycle races, Telemetry data, Race dynamics, High-granularity data, Peloton, Attack and breakaway

Resumen

El ciclismo profesional genera grandes cantidades de datos de telemetría a una alta frecuencia durante las carreras por etapas; sin embargo, las herramientas existentes ofrecen poco más que resultados finales y estadísticas básicas, dejando la dinámica y el desarrollo de la carrera en gran medida inaccesible. Este proyecto propone una aplicación web de analítica diseñada para explotar datos de telemetría por ciclista — como la velocidad, la posición en carretera, la pertenencia de grupos y las diferencias de tiempo respecto al líder — muestreados cada 10 segundos, con el fin de permitir una exploración y análisis detallados de etapas individuales. Utilizando la Etapa 7 del Tour de Francia 2021 como referencia, se ha desarrollado una interfaz interactiva con React y FastAPI, combinando Plotly y Apache ECharts para ofrecer visualizaciones ricas e interactivas que permiten examinar la carrera en cualquier momento o intervalo seleccionado y extraer un análisis del rendimiento global, por ciclista y por equipo a lo largo de toda la etapa. Evaluada frente a un conjunto de preguntas de investigación que abarcan tres niveles analíticos, la aplicación tiene como objetivo ofrecer, tanto a aficionados al ciclismo como a analistas, una herramienta potente para comprender con un alto nivel de detalle el desarrollo de cada etapa y carrera.

Palabras clave

Analítica visual, Carreras ciclistas por etapas, Datos de telemetría, Dinámica de carrera, Datos de alta granularidad, Pelotón, Ataque y escapada

Resum

El ciclisme professional genera grans quantitats de dades de telemetria d'alta freqüència durant les curses per etapes; tanmateix, les eines existents ofereixen poc més que resultats finals i estadístiques bàsiques, deixant la dinàmica i el desenvolupament de la cursa en gran mesura inaccessible. Aquest projecte proposa una aplicació web d'anàlisi visual dissenyada per explotar dades de telemetria per ciclista — com ara la velocitat, la posició en carretera, la pertinença a grups i les diferències de temps respecte al líder — mostrejades cada 10 segons, per tal de permetre una exploració i anàlisi detallats d'etapes individuals. Utilitzant l'Etapa 7 del Tour de França 2021 com a referència, s'ha desenvolupat una interfície interactiva amb React i FastAPI, combinant Plotly i Apache ECharts per oferir visualitzacions riques i interactives que permeten examinar la cursa en qualsevol moment o interval seleccionat i extreure una anàlisi del rendiment global, per ciclista i per equip al llarg de tota l'etapa. Avaluada davant d'un conjunt de preguntes de recerca que abasten tres nivells analítics, l'aplicació té com a objectiu oferir, tant als aficionats al ciclisme com als analistes, una eina potent per comprendre amb un alt nivell de detall el desenvolupament de cada etapa i cursa.

Paraules clau

Anàlisi visual, Curses ciclistes per etapes, Dades de telemetria, Dinàmica de cursa, Dades d'alta granularitat, Pelotó, Atac i escapada

Contents

1	Introduction and State of the Art	5
1.1	Multi-Stage Bicycle Races	5
1.2	Data in Modern Cycling	5
1.3	State of the Art	6
1.4	Project Motivation	7
2	Goals of the project	8
2.1	Research Questions	8
3	Data and Preprocessing	10
3.1	Data	10
3.2	Data Preprocessing	10
3.2.1	Identification and Classification Variables	12
3.2.2	Performance and Race Dynamics Variables	12
3.2.3	Environmental Conditions Variables	13
3.3	Precomputed Data	14
4	Application	15
4.1	Technologies	15
4.1.1	Backend	15
4.1.2	Frontend	17
4.2	Menu	17
4.3	Stage Overview	18
4.3.1	Overview Rankings	18
4.3.2	Key Points of the Stage	19
4.4	Stage Instant	21
4.4.1	Stage Instant Filters	21
4.4.2	Speed Context Heatmap	23
4.4.3	Race Standings	24
4.5	Stage Interval	26
4.5.1	Stage Interval Filters	26
4.5.2	Interval Heatmap	28
4.5.3	Gap Lollipop Chart and Position Slope Chart	29
4.5.4	Group Transition Sankey	30
4.6	Analytics	32
4.6.1	Gap Evolution	32

4.6.2	Rider Group Evolution Chart	33
4.6.3	Average vs Final Position	35
4.6.4	Climber vs Sprinter	36
4.6.5	Rider Performance Radar	37
5	App Evaluation	40
6	Conclusions	43
7	Future Work	44
A	Sustainability Analysis and Ethical Implications	46
A.1	Sustainability Matrix	46
A.2	Ethical Implications	46
A.3	Relation to the Sustainable Development Goals	47
A.4	Use of Artificial Intelligence	47
B	Preprocessing Charts	48

1. Introduction and State of the Art

1.1 Multi-Stage Bicycle Races

A multi-stage bicycle race (or stage race) is a single cycling event broken into multiple distinct races called stages, held consecutively over a series of days or weeks. The most well-known races of this kind are Giro d'Italia, La Vuelta in Spain and the one we are going to start with in this project: Tour de France.

Unlike a one-day classic race where the first rider across the finish line wins everything, a stage race is an ongoing competition of endurance, strategy, and recovery, in which riders face a wide variety of challenges. Over a series of consecutive days, competitors must conquer brutal mountain passes, battle solitary time trials, navigate volatile weather conditions, and manage physical exhaustion, all while fighting to survive the strict daily elimination time cuts. This gruelling format transforms the event into a dynamic war of attrition, requiring athletes to constantly balance individual stage glory with the long-term objective of maintaining a high position in the cumulative general standings.

Behind every individual rider crossing the finish line lies a highly organized team working toward a shared goal. A cycling team enters a stage race with a roster of specialists, each assigned a specific role. Sprinters burst with speed on flat finishes, climbers thrive on steep mountain passes, and domestiques (or helper riders) completely sacrifice their own chances of winning to protect their team leader: riding against the wind to shield them, dropping back to the team car to fetch water bottles, or even surrendering their own bicycle during a mechanical failure. Managing these roles is a major psychological challenge for team directors, who must convince world-class athletes to suppress their personal ambitions for the benefit of the team's collective strategy.

A multi-stage race is unique in that it hosts several entirely different competitions within the exact same event. Rather than tracking only who wins each day, a stage race runs multiple simultaneous leaderboards:

- **General Classification (GC):** the main leaderboard, calculated by adding up each rider's elapsed time from every stage. The rider with the lowest cumulative time leads the race and wears a distinctive jersey (such as the iconic yellow jersey of the Tour de France).
- **Points Classification:** rewards speed and consistency. Riders earn points for finishing near the front on flat stages or winning intermediate sprints mid-stage. This ranking is typically contested by specialist sprinters.
- **Mountains Classification:** rewards the best climbers. Points are awarded to the first riders to reach the summit of each designated categorised climb along the route; harder climbs carry more points.
- **Best Young Rider:** follows the same time-based rules as the General Classification but is restricted to riders under a specified age threshold (typically 26).
- **Teams Classification:** aggregates the daily finishing times of each team's three fastest riders per stage. The team with the lowest collective cumulative time leads this ranking.

1.2 Data in Modern Cycling

Like many elite sports today, professional cycling has undergone a significant transformation driven by data. Multi-stage races now deploy lightweight GPS and telemetry sensors on every rider throughout the entire

event, continuously recording coordinates, instantaneous speed, altitude, and timing information at high frequency. This stream of data serves two primary purposes.

On one hand, it is invaluable to the teams themselves: real-time telemetry informs tactical decisions during a stage, allows sports directors to monitor how their riders are performing under load, and provides post-race evidence to justify or revise strategic choices. On the other hand, the race organisation and broadcasters use this data to bring the action to television and online audiences worldwide, enriching live coverage with on-screen graphics showing gaps, group formations, and speed profiles.

However, what level of analytical depth is achievable with the data and resources currently available for this type of race? Let's build a scenario which is going to guide the motivation and evaluation of the project.

It is the evening of Friday, 2 July 2021. Stage 7 of the Tour de France has just finished: 249 kilometers from Vierzon to Le Creusot, with five categorised climbs and a demanding final section. A Spanish cycling fan, a supporter of *Movistar Team*, one of Spain's most storied professional squads, sits down to find out how their team performed. The stage has been won by Matej Mohorič for *Bahrain Victorious*, in a long solo breakaway. The fan wants to understand more than just the final result. They want to know: *Where was Movistar during the race? Were they ever in contention? At what point did the gap to the front group open up? How did their riders' speed compare to Bahrain Victorious on the decisive climbs? Were they sacrificing riders to protect a team leader, or did the whole team fade together?*

This is not an unusual question for a passionate fan or an analyst. And yet, as the following review of existing tools will show, no available application is capable of answering it effectively.

1.3 State of the Art

The Tour de France app by Škoda [1] is the official application of the race. It offers live tracking during the event and a post-stage summary, but its primary focus is real-time information during the broadcast. Once the stage is over, it does not provide any tools to go back and explore the dynamics of what happened. The fan in our scenario would find the final result and a highlights video and nothing more.

On the other hand, several well-established websites offer comprehensive historical race data in the form of structured results and statistics:

- ProCyclingStats [2] is one of the most complete sources of professional cycling data, covering race results, rider profiles, team rosters, and historical rankings across all major competitions.
- FirstCycling [3] provides detailed race results, stage breakdowns, and rider career statistics, with a strong focus on historical depth.
- ProTour Cycling [4] aggregates race calendars, results, and team and rider information across all major UCI tours.
- CyclingArchives [5] and CQ Ranking [6] similarly catalogue race outcomes, points standings, and rider rankings over time.

These platforms are excellent references for *what* happened: who finished where, with what time gap. However they cannot tell our fan whether Movistar was in the lead group at kilometer 180, how their

speed compared to Bahrain Victorious on the *Signal d'Uchon* climb, or at what exact moment the race was decided.

Cyclingoo [7] goes a step further than pure results. In addition to official rankings and team information, it presents data on the key points of the race, such as intermediate climbs and sprint sectors, and includes a visual stage profile. This helps the fan understand the terrain and the context, but it still offers no way to explore the dynamics of the race itself. Movistar's riders appear only as names in a results table, indistinguishable from any other team.

Tour Tracker Pro Cycling [8] extends this with detailed timing information and an option to replay the race at specific moments, including a description of group formations and notable events. This is one of the closest existing approaches to genuine race exploration. However, the level of detail here is not enough to do a deeper analysis of any rider or team. Our fan could see that Movistar was in the peloton at a given checkpoint, but not how they got there, how fast they were moving, or how they compared to the stage winner's team across the race.

Finally, TourVIS [9] is an application developed by J. Diaz, M. Fort, and P. Vázquez which effectively summarises stage information, group dynamics, rankings, and team and individual rider data, presenting a large amount of information within a single integrated view. TourVIS represents the closest academic precedent to the goals of this project. However, it shares the same fundamental constraint: it operates on low-granularity data (each 10 km in this case). Fine-grained dynamics, such as how a team's speed varied across a specific climb, or exactly when and how a group fragmented lie beyond its reach.

1.4 Project Motivation

Looking across all the tools reviewed, a common pattern emerges. They all answer the question "*what was the result?*" reasonably well. Some go further and answer "*what were the key moments?*" at a coarse level. But none of them can answer the question our Spanish Movistar fan is actually asking: "*how did my team perform, in detail, compared to the best team on the stage, at every point of the race?*"

The gap is not a matter of interface design or missing features, it is a matter of data granularity. Without high-frequency, per-rider telemetry data covering every instant of the stage, this level of analysis is simply impossible.

This project starts from a fundamentally different premise. Its central asset is high-granularity telemetry data recording the position, speed, group membership, altitude, and time gap (among others) of every rider at every instant throughout the stage — around 350,000 rows of data per stage.

The aim of this project is therefore to develop a visual analytics application for multi-stage bicycle races that enables deep and detailed exploration of a stage, placing high-granularity data at the centre of the experience, and going well beyond the basic information and summary visualizations offered by all tools currently available.

2. Goals of the project

Knowing the aim of the project, this section specifies its concrete goals. Returning to the motivating scenario introduced earlier, a Spanish Movistar fan wanting to understand in depth how their team performed in Stage 7 compared to Bahrain Victorious, the application must do far more than display a results table. It must invite exploration, support analysis, and provide answers that no existing tool currently offers.

With this in mind, the goals of the application are the following:

1. **Deep exploration and broad analysis.** The application allows the user to explore the race at any level of detail, from a high-level overview of the final standings to a granular, instant-by-instant examination of rider and group behavior throughout the stage.
2. **Exploitation of high-granularity data.** The application makes full use of the high-frequency data available to allow high level detail analysis.
3. **Balance between clarity and information density.** The interface seeks the most suitable trade-off between a clean, accessible design and visualizations that maximise the information conveyed.

2.1 Research Questions

Some of these goals are, by nature, somewhat subjective. In order to guide the construction of the application and to evaluate, at the end, how well it achieves its goals, the design process has been structured around a set of concrete research questions that the application should be able to answer.

The starting point for these questions was the set of user queries collected in the TourVis study [9], which identified two levels of questions that cycling fans and followers want to answer from race data:

- **Basic Questions (BQ)** — accessible to any user, including those with little prior knowledge of cycling. These questions deal with facts and results.
- **Specific Queries (SQ)** — targeted at followers who understand the sport and want to go beyond the result to explore the tactical and competitive dynamics of the race.

To these, a third level has been added. Knowing the granularity and richness of the data available for this project, a set of analytical questions was formulated that operate at a deeper level of interrogation than BQ and SQ:

- **Analytical Questions (AQ)** — designed for expert users and potential team analysts who require precise, quantitative insight into rider and team performance. These questions are original contributions of this work, formulated specifically to exploit the high-frequency, per-rider data that makes this level of analysis possible.

All questions across the three levels have been grouped into two principal questions, Q1 and Q2, which reflect the two main analytical axes of the application. Three further questions (Q3, Q4, and Q5) have been added as original contributions of this work, also situated in the analytical plane and motivated by the depth of insight that the available data granularity makes possible. The full set of questions is as follows:

Q1 — Behavior and evolution of a rider, team, or group throughout a stage

- BQ3** What are the time gaps between riders at a specific instant of the stage?
- BQ6** Which cyclists belong to each team?
- SQ3** Have there been attacks or breakaways during the stage?
- SQ4** What groups formed during the stage? Do they represent a threat to the race leader?
- SQ5** What is the time gap between a contender and the leader at a specific point of the stage?
- AQ1** How did a rider's position evolve over a given moment or interval of the race — on a climb, approaching the finish line, or at any other point?
- AQ3** What was the likely strategy of a team at a specific moment of the race?
- AQ6** How did a team evolved and behave across the stage as a whole?

Q2 — General and stage rankings before, during, and after the stage

- BQ1** Who won the stage?
- BQ2** What was the final finishing order?
- BQ4** Who leads the overall race after this stage?
- SQ1** Who is the virtual race leader at a specific point along the route?
- AQ2** Which riders gained or lost the most time relative to the race leader over a given interval?
- AQ4** What do the various classifications look like at a specific moment of the stage, and how do they evolve throughout?

Q3 — Team performance and strategy in a specific stage

What was a team's overall performance across the stage? What strategy might they have been pursuing? Did they execute it successfully, or did they fall short?

Q4 — Evolution of teams, riders, and groups at key moments of the stage

How did teams, riders, or groups behave during the decisive sections of the stage — the key climbs, the breakaway, the final kilometers?

Q5 — Stage characteristics and profile

What does the stage look like? What are its key points (its climbs, its sprint points, ...) and how do they shape the way the race is likely to unfold?

Together, these five question groups span the full range of what the application is designed to support: from the most basic facts, to the kind of performance analysis that only high-granularity data can enable.

3. Data and Preprocessing

3.1 Data

Let's take a deeper look at the data we have to do the application. For each stage, we have 3 types of data: Ranking Data, Stage Data and Telemetry Data.

Regarding the Rankings, this project makes use of a set of Excel files (.xlsx) extracted from the official Tour de France website [10]. These files include:

- The General Classification: The overall standings as they stood before each stage.
- The Stage ranking: The finishing order of the stage itself.
- The secondary classifications: Points, Climbers, Youth, and Teams.

Furthermore, GPX files were downloaded from *La Flamme Rouge* website [11] containing important information about the exact location (LAT, LON) and elevation (ALT) of each point of the race. This is the Stage Data we were talking about, coupled with the exact points of the climbs and sprints of the stage.

Finally, the Telemetry data was extracted from the official Tour de France website [10] using a Python script and stored in a series of CSV files. Data was collected at a sampling frequency of one snapshot every 10 seconds. Depending on the stage, which can range from approximately 3 to 6 hours in duration, this yields between roughly 1,000 and 2,200 unique instants covering the full race from the start to the last rider crossing the finish line.

At each instant, one record is stored per rider, ordered by race position. Given that a Tour de France stage typically features between 170 and 184 active riders (depending on how many retired), a single stage produces in the order of 200,000 to 400,000 rows.

Table 3.1 describes all variables available in the commented CSV of the Telemetry data. The columns are grouped into three categories: *identification and classification* variables, which describe who the rider is and where they stand in the race, *performance and dynamics* variables, which capture the racing behavior at that instant, and *environmental* variables, which record the conditions on the road.

All variables from section 3.1 were the original ones present in the dataset. However, there are variables which were added during the data enrichment phase. These variables are the following:

- INST: Constructed to provide an explicit, unambiguous identifier for each 10-second snapshot, as the raw data contained no such column.
- ALT: Computed by cross-referencing each rider's DIST FINISH value against the official GPX route file, using linear interpolation to estimate the altitude at each point along the course.

3.2 Data Preprocessing

Due to the high granularity of the data, it became clear from the outset that several columns contained noise, inconsistencies, and errors that needed to be corrected before the data could be used reliably in the application. Presenting uncleaned data to users would mean showing incorrect gaps, wrong group assignments, or impossible speed values, so directly contradicting the core goal of the project. This section describes the preprocessing steps applied to each column.

Variable	Units	Description
Identification and classification		
POS	—	On-road race position of the rider at this instant (1 = race leader).
RIDER	—	Full name of the rider.
NUM	—	Number identifying the rider in the race.
TEAM	—	The rider's professional team.
GROUP	—	Name of the on-road group the rider belongs to at this instant (e.g. Leader Group, Peloton, Chase Group 1, ...).
Performance and race dynamics		
DIST_FINISH	km	Distance remaining to the finish line. Decreases over time as the rider advances.
GAP_HEAD_OF_THE_RACE	s	Time gap between this rider and the race leader. Zero for the rider in first position.
GAP_PREV_GROUP	s	Time gap between this rider's group and the group immediately ahead on the road.
DELAY_GR	s	Time delay between this rider and the race leader in the overall classification before the stage. This value is constant.
CURRENT_SPEED	km/h	Instantaneous speed of the rider at this exact moment.
GRADIENT	%	Road gradient at the rider's current position. Positive = ascent, negative = descent.
Environmental conditions		
WIND_SPEED	km/h	Wind speed at the rider's location.
WIND_DIR	deg	Wind direction in degrees (0–360).
TEMP	°C	Air temperature at the rider's location.

Table 1: Telemetry Data variables

All preprocessing was carried out in Python using the following libraries: *pandas* for data manipulation and column-by-column transformations, *NumPy* for numerical operations and vectorised computations, *Plotly Express* for exploratory visualizations that guided the detection of each problem, and *Matplotlib* for auxiliary distribution plots.

The only stage presented in this project is Stage 7, so its preprocessing is presented here, covering the types of data quality issues that other stages may also exhibit.

3.2.1 Identification and Classification Variables

Before any column-level cleaning, a fundamental preprocessing step was required: creating the INST column itself, since the raw data contained no explicit snapshot identifier. The instant was derived using a cumulative sum over occurrences of POS = 1, each time the race leader's record appears, a new instant begins. Two instants (2 and 103) were found to be missing their position-1 record, causing rows from two consecutive snapshots to be merged into one. This was corrected by inserting artificial sentinel rows at the correct positions in the dataframe, recalculating the cumulative sum, and then removing those rows. Additionally, instant 2 contained a block of duplicate rows for positions 161–170, which were removed as they were neither correct nor recoverable.

Regarding POS it was verified that, within each instant, its values are strictly monotonically increasing and that no two riders share the same position. Both conditions were satisfied throughout the dataset. However, there were some instants which had missing data in this variable, especially in the back of the race. It was decided not to impute or recover those values because most of them were consecutive and in the back of the race, with no more context than the last available rider data in the race (which was not in the last position because there was missing data). Figure 20 in Appendix B shows a heatmap with the missing data of this variable.

Inspecting the variable RIDER it was found that two riders, Goldstein and Greipel, are absent from the first instants of the race and only begin to appear partway through the stage, indicating a loss of telemetry signal at the start. This is not flagged anywhere in the raw data and is invisible in standard results tables, illustrating the kind of data quality issue that only becomes apparent at this level of granularity. As there was no previous context of its performance till they appear in the data, no imputation or recovery was performed here either. There are also other riders who had many missing rows of their data, as it can be seen in Appendix B Figure 21. However, it was also decided for them not to recover its values because the rows were missing many consecutive instants, so it was not possible to impute all row values at once without distorting reality.

Lastly, it was verified that each rider is associated with exactly one NUM and one TEAM throughout all instants. No inconsistencies were found in either column.

3.2.2 Performance and Race Dynamics Variables

This section of variables begins with DIST FINISH (km). For this float variable, first it was checked that it contained no null or unusual values (outside the expected ones from the start to the finish of the race). Then, further analysis was performed. This column should decrease monotonically over time for each rider, as they advance toward the finish line and it has no sense to go backwards. However, 62 riders presented at least one instant where their recorded distance was greater than the instant before, which is not reasonable. These values were corrected by replacing any non-decreasing entry with the value from the previous instant, on the basis that the most conservative assumption is that the rider remained stationary rather than moved backwards, caused by a small crash or interruption, for example.

Then we have GAP HEAD OF THE RACE (s), which was the most critical column and required the most extensive preprocessing. At Appendix B, Figure 22 shows an evolution chart of this uncleaned variable, whereas Figure 23 shows its cleaned version. The fixed problems with this variable are explained below:

1. **Localised noise between km 37 and km 27 from the finish.** In this zone, the gap values of several riders fluctuate wildly from one instant to the next (variations of several minutes within a single 10-second interval) which is physically impossible. These values were corrected iteratively: any

entry in that zone where the gap dropped by more than 60 seconds compared to the previous instant was replaced with the previous instant's value, repeating until no such cases remained.

2. **Gaps collapsing to zero near the finish.** In the final kilometers of the stage, most riders' recorded gaps drop abruptly to zero for several consecutive instants before returning to their real values. This is clearly erroneous as only the leader of the race has a gap of zero. The affected segments were identified as runs of zero values in riders with a position greater than 5, and were corrected by linearly interpolating between the last correct value before the anomaly and the first correct value after it.
3. **Isolated outliers throughout the stage.** A total of 827 upward spikes and 228 downward spikes were detected across the full dataset, points that differ substantially from both their preceding and following values while those neighbours remain consistent with each other. These were imputed iteratively by replacing each outlier with the mean of its two neighbours until no further outliers were detected.
4. **Inconsistent gap ordering within an instant.** Since riders are ordered by position within each instant, the gap to the leader must be non-decreasing as position increases. Approximately 3,500 small violations (between 1 and 10 seconds) and 987 larger violations were found. Both were corrected by replacing the offending value with the gap of the rider in the position immediately above.

Two issues were found in GROUP variable. First, one rider had a group value of '-', which was manually reassigned to *Peloton* based on their position at that instant. Second, and more systematically, some instants presented intercalated group sequences: a group appearing, then a different group, then the first group again, within the same instant. Since riders in the same group must appear as a contiguous block ordered by position, these intercalated cases were corrected by extending the surrounding group to cover the isolated entries.

GAP PREVIOUS GROUP (s) records the time gap between a rider's group and the group immediately ahead of it. It should satisfy two conditions: all riders within the same group at the same instant must share an identical value, and that value must equal the difference between the minimum GAP HEAD OF THE RACE of the current group and the minimum of the preceding group. Both conditions were violated in a large number of cases. An automated correction function recalculated all values from scratch using the cleaned GAP HEAD OF THE RACE column.

While CURRENT SPEED (km/h) is one of the most important variables in the dataset, as it is widely used in the presented analysis, it presents no inconsistencies. First it was checked that there were not physically impossible outliers looking at the extreme values from this values. Those values were 0 and 107, which are correct taking into account possible falls or interruptions with the minimum and high speeds in the descents for the higher one. Here it was also checked for possible noisy or outlier values as the GAP HEAD OF THE RACE but no important cases were found here. Appendix B Figure 24 show the distribution of all values in this variable, showing no abnormalities as commented.

Lastly, no strange values were found for Gradient (%) and for Delay GR (s) it was verified that each rider has a constant value for this column across all instants. The condition was satisfied, so no corrections were necessary.

3.2.3 Environmental Conditions Variables

Wind direction (WIND DIR (deg) variable) should always fall within the range 0–360 degrees. A total of 11,454 rows were found with values outside this range. These were corrected by applying a modulo-360

operation, mapping all values back into the valid interval.

Lastly, both columns `WIND SPEED (km/h)` and `TEMP (°C)` contained a small number of NaN values. These were filled in two stages: first using the mode of the corresponding instant (the most common value among all riders at that moment), and then, for any instants where all values were null, using the global mode of the entire column.

3.3 Precomputed Data

Several analytics charts require aggregations over the full telemetry dataset as the large quantity of data made it too expensive to compute them on demand. All such transformations are performed offline by a dedicated Jupyter notebook script, which queries the PostgreSQL database, aggregates and encodes the data for each specific chart, and writes the results as static JSON files. These files are then served as static assets and fetched by the frontend at page load, so the application requires no real-time database computation.

4. Application

Once we have all the ingredients, such as motivation, goals and data, it is time to explain the proposed solution. First we are going to explain the overall idea and the technologies, which are justified with the proposed solution and were thought on the go to solve the problems we faced.

In order to answer as many questions as possible effectively, it was decided to distribute the app into 4 distinct pages. The pages are ordered from low-level knowledge to high-level analysis, so all type of users can inspect whatever they want and all of them find the information they want rapidly.

The *Stage Overview* page is intended to show general information and context about the race. This page gives context and also gives basic information for those who don't know what happened in that stage.

The *Stage Instant* page focuses on the exploration part of the application as it shows specific information for each instant of the race in order to inspect whatever part of the race we want to see and analyze precisely.

The *Stage Interval* page trades off explorability and analysis, as it allows to filter by interval of instant in the range and see what happened in those parts. This allows deeper analysis than the page before.

Finally, the *Analytics* page focuses on different specific analysis of the race which were thought to enrich the application.

4.1 Technologies

To explain the technologies, a Box Diagram (Figure 1) is going to be used in order to guide and justify the explanation. The box not only shows the technologies used but also the overall pipeline that the data follows from the source to the application.

The application begins with two external data sources, visible at the very top of the diagram. On the left, the Tour de France official website provided the telemetry tracking data for every rider, which was scraped with a Python script as commented in previous sections and it is shown in the diagram. On the right, La Flamme Rouge supplied the GPX file for the stage route, which encodes the road as a sequence of GPS coordinates with elevation data.

Then we get to the preprocessing, which was done in python using several Jupyter notebook files. Here it was performed all the preprocessing commented in Section 3.2 and the creation of the precomputed data to feed the charts that do not depend on the PostgreSQL data.

This step introduces a question:

Why there is some data stored in a database and other data in JSON files?

4.1.1 Backend

To answer this we have to take into account the proposed solution. On the one hand, we have the pages *Stage Overview* and *Analytics* which display fixed, pre-aggregated analysis of the race data that do not change depending on user interaction. The *Stage Overview* page shows the official classification tables, which are static by nature. The *Analytics* page displays charts which show information that is always computed over the entire stage regardless of what the user selects. For this reason, the data for these pages was pre-computed offline using Python Jupyter notebooks and stored as static JSON files served directly from the application. These files are fetched by the browser at page load without any database query, making both pages load instantly.

Visual Analytics Application for Multi-Stage Bicycle Races

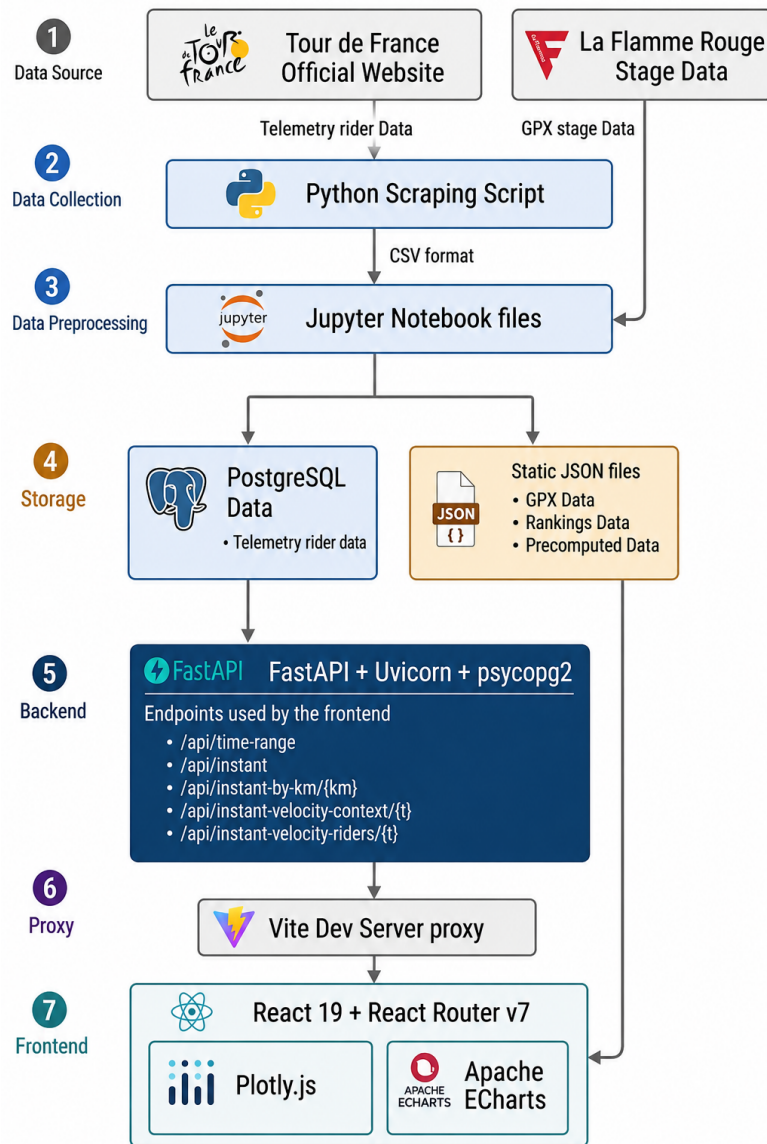


Figure 1: Technologies Box Diagram

On the other hand, *Stage Instant* and *Stage Interval* need to be able to show information from any of the approximately 300,000 rows on demand, as they are designed to allow deep exploration. Loading all of this data into the browser at once is not feasible: it would consume large amounts of memory, take a long time to transfer, and make filtering slow. For this reason, the solution was to use a database to store all the data and a framework to query it on demand.

As shown in Figure 1, the database chosen for the development phase is *PostgreSQL*, as it has been widely used throughout the degree and it offers all the functionality required for this project: advanced SQL querying capabilities and reliable structured data storage.

FastAPI was used to expose the database data through a set of API endpoints, each returning different information depending on the query. These endpoints will be described in more detail in the following

sections, when the functionalities of each page are explained. The use of *FastAPI* would not be possible without *Uvicorn*, the server that listens for HTTP requests and forwards them to the application, nor without the *psycopg2* driver, which handles the actual connection and query execution against the PostgreSQL database.

4.1.2 Frontend

The frontend of the application is built with *React 19*, a JavaScript library for building user interfaces, combined with *React Router v7*, which maps each URL path to a different page component. This allows the user to navigate between the pages of the application without any full page reload, resulting in a fluid experience. It was decided to use React instead of other frameworks such as Streamlit or other options because one of the priorities was full control and personalization over the frontend, to achieve a pleasing visual interface which allows easy navigation and understanding of the app by the users.

For data visualization, two complementary libraries are used. *Plotly.js* handles interactive charts that require precise axis control and zoom, such as the elevation profile and scatter plots. *Apache ECharts* is used for heavier visualizations such as the speed heatmaps and the parallel coordinates chart, all of which will be explained in the following sections. These two are among the most widely used libraries for interactive visualizations, which is the reason they were chosen for this project.

In order for the browser to communicate with the *FastAPI* backend without issues, a proxy layer is needed during development. By default, browsers block requests made to a different server than the one that served the page, as a security measure. *Vite*, beyond acting as the development server that runs the *React* application, also includes a built-in proxy that intercepts requests directed to the *FastAPI* backend and forwards them transparently, returning the response to *React* as if it had come from the same origin.

4.2 Menu

The application opens on the Menu page, which serves as the main entry point to the platform. From here, the user selects which race and stage they want to explore before accessing any of the visualization pages. This page is shown in the Figure 2.



Figure 2: Menu page

Here, the web application presents itself as the **MULTI-STAGE BICYCLE RACE LAB**, and the page structure here is simple: left column to select the race, and once a race is selected, its stages are presented on the right. Each race/stage box has brief context information.

At the moment, the only race available is the 2021 Tour de France Race and the only stage available is Stage 7. As mentioned, this project focuses on optimizing the visualization and analysis of this type of races, so for this reason it was decided to focus only on one stage for that purpose.

Once we select a stage, the first page we see is *Stage Overview*, navigation to the other pages is allowed at the top of the pages with the navigation bar shown in Figure 3.

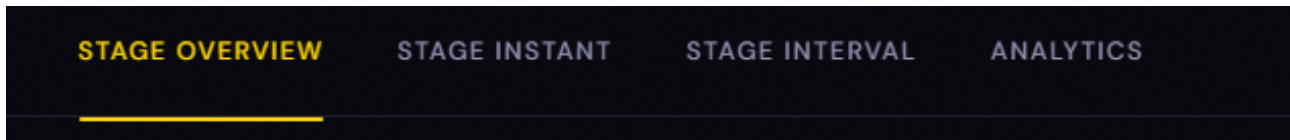


Figure 3: Navigation Bar

4.3 Stage Overview

Once a race and stage is selected, the first page that comes into action is the *Stage Overview*, a page which provides a complete static summary of the stage. As mentioned, all the information displayed here is loaded from JSON files, created from the official race classifications of the official Tour de France website [10] and the stage route data from *La Flamme Rouge* [11], so no live queries are made to the database here. This page is designed to give the user an immediate, at-a-glance understanding of how the stage unfolded and what the final standings looked like, in addition to the context of the key points of the race.

4.3.1 Overview Rankings

As shown in Figure 4, the top of the page presents a stage hero block that condenses the essential facts about the stage: the route, total distance, terrain type, the number of categorised climbs, the location of the intermediate sprint, the name of the stage winner, and the rider who led the general classification after the stage.

Below the hero block, the page displays seven classification panels organized in two rows. Each classification or ranking presents its top-3 riders ordered by position, presenting their full name, the team they belong to, and the corresponding metric of the ranking which will be explained coming up. Furthermore, each rider/team row is colored by its team color. This is done to give more context and information to the user, in order to guide its decision of filtering in the following pages.

The first row contains three panels that capture the key standings around the stage:

- **General Ranking before the Stage:** Aggregated time of the riders through the race up to the beginning of the presented stage
- **Stage Ranking:** Final Time of the riders in the presented stage.
- **General Ranking after the Stage:** Aggregated time of the riders through the race including the presented stage

The second row contains four panels covering the secondary rankings. Each one of these rankings is colored by the typical color of the maillot awarded to the winners of this categories at the end of the race.

- **Points classification** (green jersey): awarded based on points scored at the stage finish and at the intermediate sprint.
- **Mountains classification** (polka-dot jersey): points earned by being among the first riders over each categorised climb; so tougher climbs are worth more.
- **Youth classification** (white jersey): the overall time ranking restricted to riders aged under 26.
- **Teams classification**: the combined time of each team's three best riders per stage, accumulated over the race.

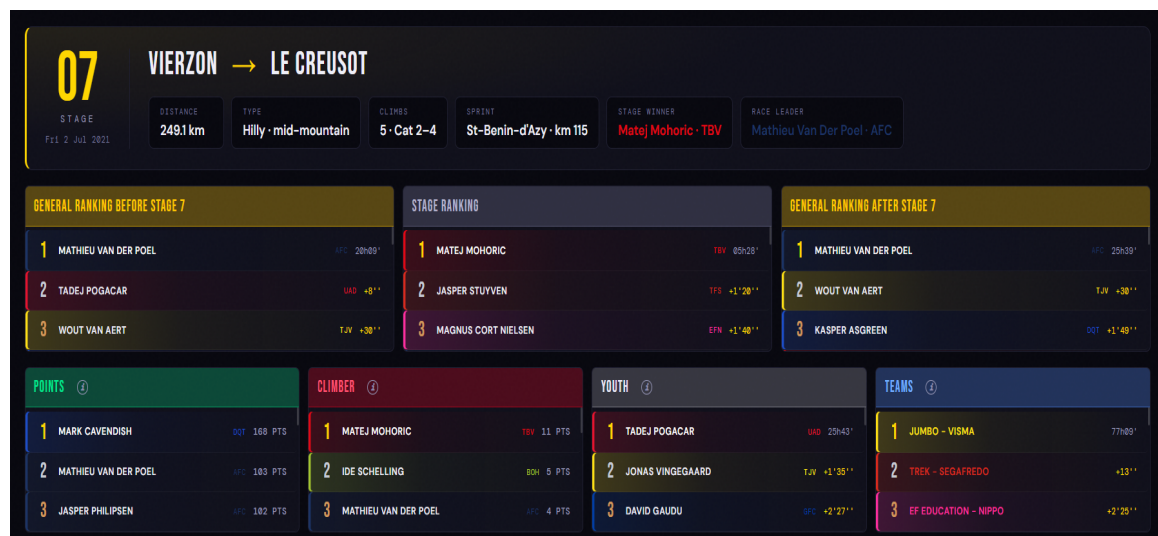


Figure 4: Stage Summary and Classifications

Unlike the other pages, the Overview rankings do not use any charting library. All visualizations are rendered as structured HTML components directly in React, styled to resemble compact leaderboard tables. The data originates from a single static JSON file fetched at runtime, which contains three ranking snapshots: the general classification before the stage, the stage race result, and all post-stage classifications (general, points, mountains, youth, and teams). The ranking data is loaded once when the page mounts and passed down to all panels as a single prop, so every component reads from the same in-memory object with no redundant network requests.

Initially, our Movistar fan came with some questions to solve, such as comparing his team performance to the stage winners. However, these visualizations allow him to think of new analysis, such as *how Movistar riders performed compared to the top 3 of the stage or general ranking, or how Movistar Team ranked compared to the top 3 teams of the stage.*

4.3.2 Key Points of the Stage

Then Figure 5 shows the Stage Profile chart, which appears at the bottom of the page. This is an elevation graph plotting altitude (m) against distance from the start (km), covering the full stage.

The profile includes two types of annotated points of interest:

- **Climbs:** the five categorised ascents of the stage are marked with colored triangles at their summits. The color of each triangle indicates the category of the climb: Category 4 climbs are the least severe, Category 3 more demanding, and Category 2 the hardest present in this stage. For each climb, a dotted vertical line rises from the profile to a label showing the name of the ascent and its category. The labels are staggered at three different heights to prevent them from overlapping in the dense final section of the route.
- **Intermediate Sprints:** the intermediate sprint point is marked with a green diamond and its own annotated label. In this case, it is the Saint-Benoît-d'Azy (km 115).

Hovering over any point on the elevation line shows the exact kilometer and altitude. Hovering over a climb triangle or the sprint diamond displays the full name, category or description, and precise location.

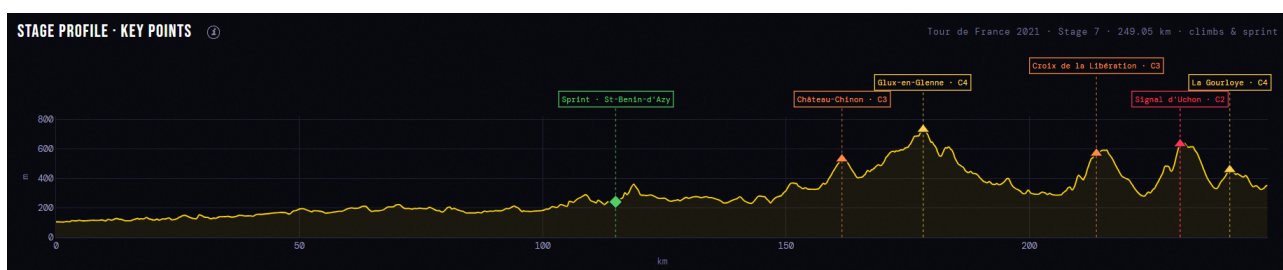


Figure 5: Climbs and Sprints

This visualization allows our Movistar fan to know the context profile of the stage, the key points where to expect action, and where to set the filters in the following pages in order to perform richer analysis.

To construct this visualization, two types of Plotly charts were used:

- **Elevation Scatter Plot:** A scatter trace in *lines* mode with a *tozeroy* fill, which shades the area between the elevation curve and the horizontal axis. A spline interpolation is applied to smooth the curve between the sampled GPS points, giving a fluid and visually clean profile. The x-axis represents distance from the start in kilometers and the y-axis represents altitude in metres.
- **Key Points Scatter Plot:** Two additional scatter traces in *markers* mode are overlaid on the elevation line. Categorised climbs are represented by upward-pointing triangles, colored by difficulty category, positioned at the summit kilometer with the elevation interpolated from the GPX data at that point. Intermediate sprints are represented by diamonds in a distinct color. Both sets of markers are accompanied by vertical dotted guide lines and floating text annotations rendered in Plotly's paper coordinate system, which keeps them anchored above the plot area regardless of zoom.

Both traces share the same data origin: the elevation profile is loaded at runtime from a static JSON file containing the sampled GPS coordinates, while the climbs and sprint locations are loaded from a separate static JSON file holding the points of interest for the stage. The elevation at each marker position is not stored directly but computed on the fly via linear interpolation over the GPS samples.

Each important chart in the application includes a small information button ⓘ that, when hovered, displays a contextual tooltip explaining the key aspects needed to interpret it. For example, the commented rankings in *Stage Overview* have its own button explaining how its classification works, as it is shown in Figure 6. This pattern is applied consistently to all significant charts throughout the application, so that users can understand each visualization without requiring prior knowledge of cycling conventions.

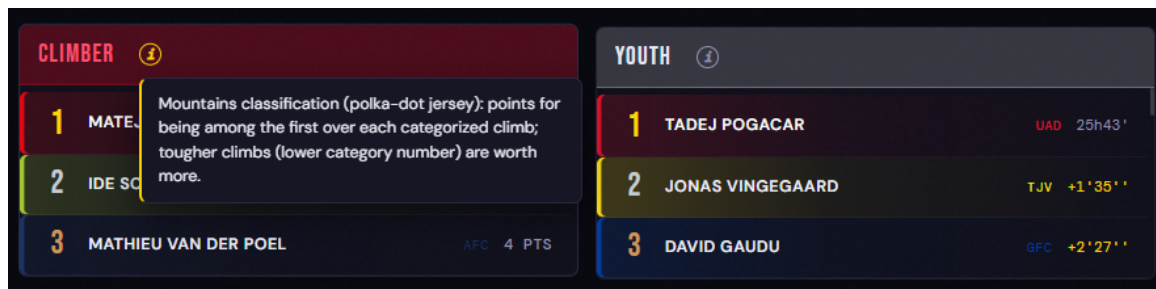


Figure 6: Information Button in Stage Overview

4.4 Stage Instant

The Stage Instant page is the exploration core of the application. It answers a very important question: *What was happening at any precise moment during the stage?*

For our Movistar fan, this is the page where the real investigation begins. Rather than looking at a final result table, the user can scrub through the entire race second by second, watching the gap between Movistar and Bahrain Victorious evolve in real time, seeing exactly when the decisive split happened, and understanding where on the road each rider was at every instant.

4.4.1 Stage Instant Filters

As you enter the page, you find the section of filters and the elevation chart of the stage, which is the core of the page. The first decision is to select the view, which selector is located at the top-left of the page, as shown in Figure 7.

This page allows two types of views:

- **General View:** Presents the information from the groups point of view and shows exact per-rider classifications.
- **Teams View:** Presents the information from the teams point of view, allowing to select up to 2 teams and compare them.

Once the View is selected, it is time to select our interested instant. The filtering approach was thought to addresses a key challenge: filtering by location ("Where") rather than time ("When"). Since the underlying data is sequenced by time instants, it is much more intuitive for users to filter by familiar race landmarks, like climbs or sprints, than by specific timestamps. Hence, three types of filtering are allowed:

- **Slider:** Allows to select an exact instant knowing at which approximate point (km) in the race we are filtering. When the slider moves, a dotted line moves with it. This line tells the user to decide 'where' he wants to filter and when the slider is dropped the filtering initiates in the background. At this point an exact km of the race is filtered, which is used to retrieve from the database the instant where the last rider is closer to that point. Then, this instant is retrieved and the page data is updated accordingly. The last rider available is selected as reference in order to be able to filter completely till the end of the race, as filtering otherwise would give problems in the final stages.
- **Step Buttons:** Allow to explore the next or previous instant from where the filter is at that moment.
- **Play:** Allows to reproduce automatically the exploration instant by instant from the point where the filter is at that moment.

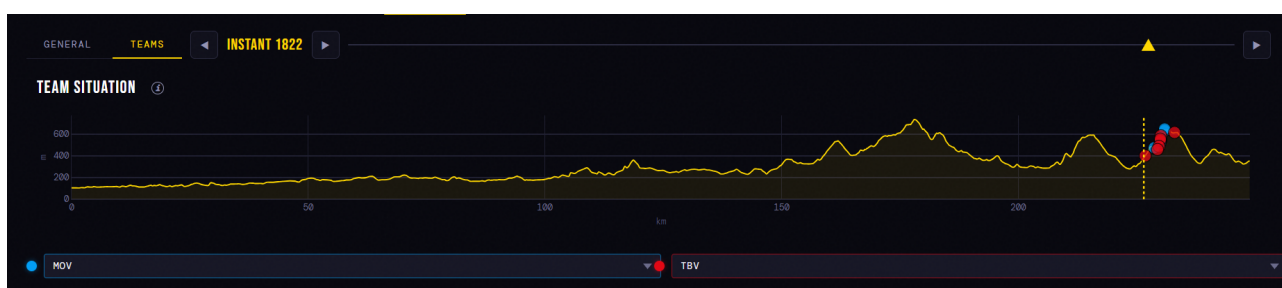


Figure 7: View Selector, Instant Filter and Stage Profile

Figure 7 shows a filtered instant which our Movistar fan may do: filtering at the beginning of the most critical climb of this stage (*Signal d'Uchon*) which is shown in the *Stage Overview*.

As it can be seen, when filtering the elevation chart situates a point per rider (colored by team) to be able to situate each member of the team in the race and know its distance to finish and altitude. Similarly, the general view shows a point per group all in the same color, in order to situate each group and know how far away they are.

The elevation chart is built with *Plotly.js*, rendered into a fixed-height container managed by a *React* ref. The base elevation trace is identical in construction to the Overview elevation chart. Then, a second scatter trace in *markers* mode is overlaid dynamically on each instant update to show the team rider or the race groups. Finally, the vertical dotted line shape is anchored in data coordinates on the x-axis in order to mark the current race kilometer on the profile and retrieve the instant depending on it.

The control bar is built entirely with *React*. As commented, it contains the view toggle, the step buttons, the play/pause button, and the kilometer slider. The slider operates in two phases: while dragging it moves the position line and markers immediately without firing a network request; on release it calls the API to load the full rider snapshot at the selected kilometer.

The elevation base trace is loaded once at page mount from the static GPX JSON file. The position markers and the vertical line are derived from the live API response for the current instant, which returns one record per rider containing their on-road position in kilometers from the finish, their GPS altitude, their race time gap, their group identifier, and their team. The kilometer-from-start value plotted on the x-axis is computed by subtracting the distance-to-finish from the total stage length.

4.4.2 Speed Context Heatmap

Besides the situation of the riders in the sage, one of the most interesting things to look at any instant is the speed of the riders, which allows the user to understand better what is happening at that moment. To solve this, the Speed Context Heatmap was created. As shown in Figure 8 the heatmap displays the speed in km/h of team riders across a context window of instants surrounding the currently selected one. The x-axis lists the instants in the window, while the y-axis lists the riders belonging to the two selected teams, sorted by their current race position. Each cell encodes the speed of that rider during that instant as a color value. Rider names on the y-axis are colored by team to maintain visual separation between the two selected squads. The selected instant is visually distinguished: its x-axis label is rendered larger and in yellow, and a floating annotation appears above the corresponding column, letting the user see not just the current speed of each rider but also whether they are accelerating, decelerating, or stable over the surrounding window. In General view the same structure applies at group level, where each row represents an on-road group and the value is the mean speed of all riders in that group.

The color palette is the nine-class *YlOrRd* scale from *ColorBrewer* [12], from pale yellow at low speeds through orange to dark red at high speeds. This sequential palette is perceptually ordered and works well on a dark background, making speed differences between riders immediately legible. The same palette is shared with General view so the color meaning is consistent across both modes.

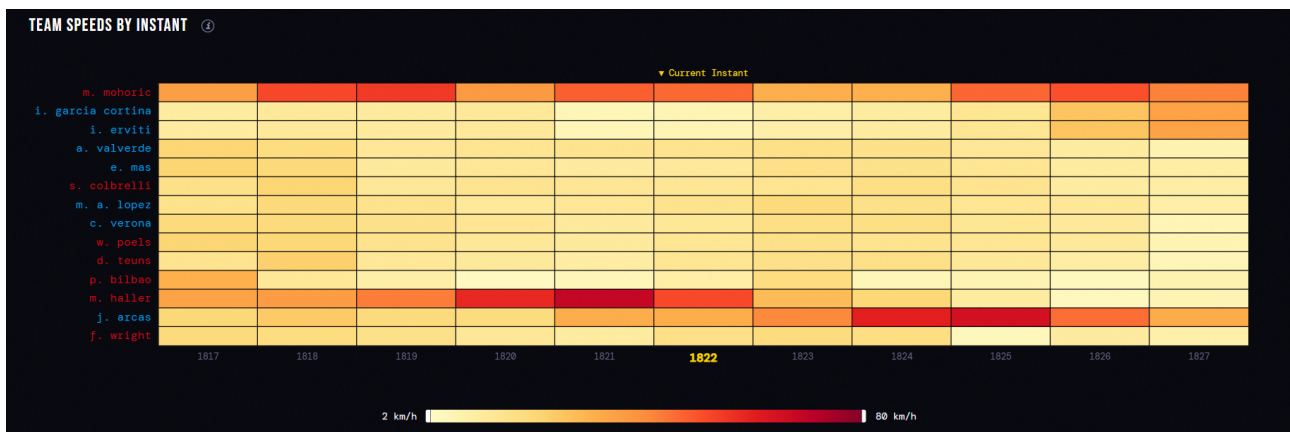


Figure 8: Speed Context Heatmap

Figure 8 shows specifically the heatmap data at the instant the Movistar fan selected in the filter as shown in Figure 7. Here, the fan is able to see which is the exact relative order of riders of these two teams and inspect their velocities at the climb of *Signal d'Uchon*. He can see that *M. Mohoric* is going visibly faster than the rest because he has already reached the summit and is beginning the descent, getting more separation from the following riders, which are mainly from *Movistar Team*. The next zone of riders till *P. Bilbao* are all really closer, this is why they present similar color encodings in the heatmap, all in a softer shade as they are ascending. *M. Haller* and *J. Arcas* present some high speed instants as they are marked with a strong red tonality. This is due to the fact there is an intermediate descent in the climb. With the hover, we can inspect deeply this part to see the exact velocities, and conclude in this case that *M. Haller* was a bit faster through that part. Then there is *F. Wright*, who is the last positioned between these two teams, and is the only one at the beginning of the ascent (before the intermediate descent).

The filter below the chart is a custom dual-handle range slider. Each handle controls one bound of the active speed range; any cell whose value falls outside the selected interval is grayed out. This allows the user to isolate, for example, only the fastest cells and immediately identify which riders were at race-winning pace and at which instants, or conversely to highlight who was already losing ground. In General view the same filter operates on group averages instead of individual rider speeds.

The heatmap is built with *Apache ECharts*, initialised imperatively into a div via a *React* ref. The chart instance is created once and reused across instant changes and team selection changes via `setOption` with the `replace` flag, avoiding full re-initialisation on every update. The key ECharts options used are:

- **Series type heatmap:** Receives a flat array of `[x, y, value]` triples, one per cell, where `x` and `y` are category indices and `value` is the speed in km/h. The flat encoding is required by ECharts rather than a nested matrix, so the data is flattened via a nested iteration before passing it to the chart instance.
- **markArea:** A zero-width area covering only the current instant column, used to render the floating *Current Instant* annotation above it. This approach was chosen over a `markLine` because an area element accepts a label anchored above the column without interfering with the cell color encoding underneath.
- **Rich text labels:** The y-axis formatter uses ECharts rich text syntax to color each rider's name with their team color, and the x-axis formatter renders the current instant label in bold yellow while all other labels remain in the standard muted style.

In General view the construction is identical except that the y-axis labels are group identifiers and no rich text coloring is applied, since groups do not carry a team identity.

The Teams heatmap is driven by a dedicated API endpoint (`/api/instant-velocity-riders/{t}`) that accepts the current instant identifier and returns a response containing the surrounding window of instants, and for each rider in the race their name, team, and the vector of speeds across that window. The frontend filters this rider list to only those belonging to the two selected teams and sorts them by their current race position before passing them to the chart. The matrix is then flattened into the flat triple array that ECharts expects via a nested iteration over riders and instants. Switching team selection does not trigger a new network request: the full rider dataset for the current instant is cached and only the filtering and chart redraw are repeated. In General view a separate endpoint (`/api/instant-velocity-context/{t}`) is called instead, which returns the same window structure but pre-aggregated by on-road group, so the mean speed per group per instant arrives ready to use with no client-side averaging required.

4.4.3 Race Standings

The filter explained in 4.4.1 shows either groups or team riders (up to two teams) situation in the race visually. However, another chart is necessary to understand the exact metrics and rankings at a glance, without needing to spend time hovering. This problem is solved by the Race Standings.

In this section, the information is shown differently depending on the General and Team view. On the one hand, Team view (Figure 9) shows all riders of the two teams, sorted by position in the race showing its GAP HEAD OF THE RACE and the group they are at that instant.

MOV			TBV		
19	I. GARCIA CORTINA	Gr.Chase Group 6 +3'02''	1	M. MOHORIC	Gr.Front of the Race Leader
20	I. ERVITI	Gr.Chase Group 6 +3'02''	44	S. COLBRELLI	Gr.Peloton +6'15''
32	A. VALVERDE	Gr.Peloton +6'15''	64	W. POELS	Gr.Peloton +6'19''
33	E. MAS	Gr.Peloton +6'15''	65	D. TEUNS	Gr.Peloton +6'21''
49	M. A. LOPEZ	Gr.Peloton +6'17''	83	P. BILBAO	Gr.Stragglers Group 3 +7'20''
59	C. VERONA	Gr.Peloton +6'19''	97	M. HALLER	Gr.Stragglers Group 4 +8'12''
101	J. ARCAS	Gr.Stragglers Group 5 +8'55''	174	F. WRIGHT	Gr.Stragglers Group 9 +13'22''

Figure 9: Team Riders Classification from Teams View

Our fan can see here that *M.Mohoric* is the leader of the race in this instant and *I.Garcia Cortina* and *I.Erviti* are the closest Movistar representatives, situated in *Chase Group 6* with a GAP of approximately three minutes. He may also find it interesting that *Movistar Team* has its riders in a more compact distribution (most of them in the peloton, apart from the 2 chasers and one straggler) while *Bahrain Victorious* is more sparse in the race at this instant, having *M.Mohoric* leading without any help.

However, it is also interesting to see the tables in the General view (Figure 10), which presents different components than Teams view, now looking at the overall race and not a subset of riders. This components are the following:

- **Stage Standing:** This is the exact stage standing at that instant, allowing scrolling to see the complete list. Again, ordered by position, colored by team and showing the group and gap head of the race.
- **Virtual General Classification:** The general classification of the stage in that specific instant. Answers the following question: *How would be the general classification if the race ended at this instant?*
- **Groups:** Filter to select the group and pannel to show the integrants of each group. This is important as selecting General View only shows the groups in the previous charts, being this the only component that allows inspecting who formed each group.

RACE STANDING ⓘ				VIRTUAL GC ⓘ				GROUPS ⓘ	
32	A. VALVERDE	MOV	Gr.Peloton +6'15''	15	E. MAS	MOV	+6'50''	Group Front of the Race	
33	E. MAS	MOV	Gr.Peloton +6'15''	16	W. KELDERMAN	BOH	+6'50''	1	M. MOHORIC TBV
34	J. ALAPHILIPPE	DQT	Gr.Peloton +6'15''	17	B. MOLLEMA	TFS	+7'00''		
35	B. MOLLEMA	TFS	Gr.Peloton +6'15''	18	J. FUGLSANG	APT	+7'10''		
36	K. ELISSONDE	TFS	Gr.Peloton +6'15''	19	D. GAUDU	GFC	+7'27''		
37	J. BERNARD	TFS	Gr.Peloton +6'15''	20	S. A. HIGUITA	EPN	+8'00''		

Figure 10: Stage Standing, Virtual Classification and Group detail from General View

Here, a different point of view analysis could be done by the Movistar fan. Via scrolling, he could see how *E.Mas* is located at the 33rd position, in the peloton, while he keeps being the better positioned Movistar Team rider in the General clasification at that instant. This means, that the better performing rider from Movistar till this instant is not performing that well at this moment in the stage or, at least, he is not performing as well as its two teammates which are in a chase group and closer to the leader. However, this is useful to see an exact photograph at a specific instant, but the whole analysis can't be done without looking to the next pages, which are more analytical. Therefore, no solid conclusions can be drawn here about the situation of *E.Mas* before checking *Stage Interval* and *Stage Overview*.

The three ranking panels in the General view and the two team panels in the Teams view use no charting library at all. They are pure *React* HTML components, following the same approach as the Overview page rankings described earlier. The Groups panel adds a native dropdown populated with the group identifiers present at the current instant. Changing the selection filters the member list below it without any network call.

All panels read from the same `lastRiders` array, which is populated on every instant load from the `/api/instant` (endpoint for step and play buttons) or `/api/instant-by-km` endpoint (for the slider). The response contains one record per rider with their on-road position, time gap in seconds, group identifier, team, and rider number. The array is sorted by position once on arrival and shared across all panels.

The Race Standing panel uses this array directly with no further transformation. The Groups panel derives a lookup map from the same array by grouping riders under their group identifier field. The dropdown options and the member list are both computed from this map on every render, so switching group is instantaneous. The Virtual GC (Virtual General Classification) panel requires a join between two data sources. The pre-stage general classification times, loaded once at page mount from the static rankings JSON file, are stored in a ref keyed by rider number. For each rider in the current instant snapshot, the page sums their pre-stage accumulated time with their current time gap in seconds to obtain a virtual total, then sorts all riders by that total ascending. The gap displayed in the Virtual GC is the difference between each rider's virtual total and the virtual leader's total, so it answers the question of who would be leading the Tour if the stage ended at that exact moment. The two team panels in Teams view simply filter `lastRiders` by the selected team name and sort by time gap, requiring no additional data or computation.

4.5 Stage Interval

While the Stage Instant page reveals the state of the race at a single moment, the *Stage Interval* page shifts the question: *What changed between two moments?* The user selects a start instant and an end instant, and every chart on the page compares those two snapshots to expose the evolution of the race within that window.

For our Movistar fan, this is the page that answers questions such as: did Movistar gain or lose time relative to Bahrain Victorious during the final climb? Which riders were promoted into the lead group over those kilometers, and which ones fell back?

4.5.1 Stage Interval Filters

Again, in this page it is of high interest to see "Where" we are filtering instead of "When". At the top of the page sit two parallel sliders (Figure 11): Start Instant and End Instant. Together they define the

interval to analyse. As either slider is dragged, the elevation profile below updates in real time the same way it does in *Stage Instant*: a semi-transparent yellow band appears over the stage profile, bounded by two dotted vertical lines that mark exact kilometer positions. This allows to paint all the zone affected by the two filtered instants. When dropping any selector, the API retrieves the instant from the kilometer, having the last positioned as reference. This visual feedback makes it straightforward to align the instant filters with meaningful sections of the race, such as a mountain pass or a sprint zone, before releasing the slider to load the data. This is the case we have in Figure 11, where the entire climb of Signal d'Uchon had been selected by the Movistar fan, in order not only to see an exact instant but analyze the whole climb.

Below the elevation profile, a global filter bar (also Figure 11) controls which riders appear in the charts. It offers three modes:

- **Top 20 finishers:** The 20 riders with the best final stage result are selected automatically. This is the default view and requires no interaction.
- **Individual riders:** A searchable checklist allows the user to select any combination of up to 20 riders from the full roster. A live text filter narrows the list as the user types.
- **Teams:** Up to 3 teams can be selected simultaneously via color-coded chip buttons. All riders belonging to the selected teams are included in every chart.

Switching between modes or changing the selection immediately redraws all charts.



Figure 11: Stage Interval Filters

The filter section contains no charting library. It is built entirely from *React* HTML controls: two native range sliders, a three-button mode toggle, a text search input with a scrollable checkbox list, and a team picker. The elevation chart underneath the sliders is the same Plotly instance described earlier, reused here to show the affected stage zone.

The slider range bounds come from `/api/time-range`, called once at page mount, which returns the minimum and maximum instant identifiers available in the database. These two values are set as the min and max of both sliders. This way, the page does not allow the Start Instant to select an equal or larger instant than Final Instant and vice versa.

The roster of riders shown in the individual picker is not fetched from the live API. Instead, it is derived from the static `interval_speed_heatmap.json` file that provides data in the next chart. This avoids a dedicated roster endpoint and keeps the picker populated immediately on page load, ensuring all riders appear and are independent of the selected instant (which could miss some rider data).

When the user releases either slider, the page gets both instant kilometer values of the stage. The kilometer value of the moved slider is then sent to `/api/instant-by-km/km` which resolves the query by returning the closest real instant in the database using the last-placed rider as the reference, as described earlier. The responses contain both the resolved instant identifier and the full rider snapshot for that instant, so no further call is needed. The elevation zone is then updated with the exact kilometer positions of the two resolved instants.

4.5.2 Interval Heatmap

When analyzing intervals, speeds are also high-interest metrics to see its evolution. Therefore, another Heatmap (Figure 12) was decided to use in this page, but this follows a different approach, as it shows average speed of each selected rider across the stage, aggregated into 3 km buckets. The y-axis lists riders sorted by final race position, with rider names colored in their team's official color. The x-axis shows the kilometer centre of each bucket. The chart is automatically cropped to the km range covered by the selected interval, so the user always sees only the relevant portion of the route.

The same *YlOrRd* sequential palette (Yellow–Orange–Red, 9-class, ColorBrewer [12]) is used as in the Stage Instant heatmaps, with the same draggable range legend at the bottom. This consistency across pages means a user switching between Stage Instant and Stage Interval can immediately interpret the color encoding without re-learning it.

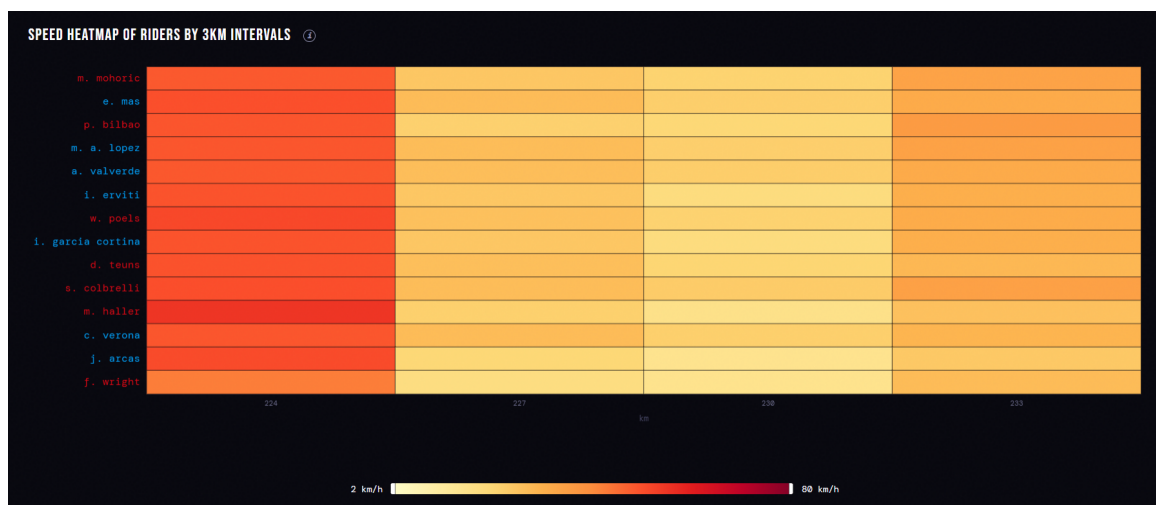


Figure 12: Stage Interval Heatmap

From Figure 12 it can be seen how Movistar Team and Team Bahrain Victorious riders developed in the climb. The two central columns reflect the steepest parts of the climb, which are not aggregated together with the exact climb point nor the descent previous to this climb (as is the case for the others). In this part, similar colors and intensities can be seen as all riders were ascending. However, we can see some cells that are darker than others, as is the case for *E.Mas*, who achieved an aggregate speed of 28.8 km/h, as shown by the tooltip. This value is the best among both teams, and followed closely by other teammates like *M.A.Lopez* with 28.5 km/h. This gives more context about the information we saw in the Stage Instant part, and shows how *E.Mas* is pushing in this part and wants to close the Gap with the leader riders, even though he had not been at his best performance until that point.

The heatmap is built with *Apache ECharts*, initialised imperatively into a div via a *React* ref. The chart instance is created once on page load and updated on every filter or interval change via `setOption` with the `replace` flag, avoiding full re-initialisation on every update. The key ECharts options used are:

- **Series type heatmap:** Receives a flat array of `[x, y, value]` triples, one per cell, where `x` is the bucket column index, `y` is the rider row index, and `value` is the average speed in km/h. This is the same encoding used by the Stage Instant heatmap.
- **Rich text labels:** The y-axis formatter colors each rider name in their team's official color using ECharts rich text syntax, consistent with the Stage Instant heatmap.
- **Dynamic chart height:** The container height is computed at draw time as a function of the number of active riders so the row height remains legible regardless of how many riders are selected. The ECharts instance is resized accordingly each time the height changes.

The draggable speed legend at the bottom is the same component used in the *Stage Instant* page. It overlays two range inputs over a gradient bar and dispatches a `selectDataRange` action to the ECharts instance on each input event, filtering out cells outside the active speed interval without repainting the full chart.

Unlike the heatmap in the *Stage Instant* page, this one is built from a precomputed static JSON file (`interval_speed_heatmap.json` as commented before) that contains the full speed matrix for every rider across the entire stage. This design decision is deliberate: computing per-km average speeds in real time for all riders across thousands of instants would be too slow for an interactive query. Instead, the precomputed matrix is loaded once when the page opens, and the interval selection simply slices the relevant column range from the already-available data.

4.5.3 Gap Lollipop Chart and Position Slope Chart

Then we get to the Gap Lollipop Chart and Position Slope Chart, which are placed side by side as shown in Figure 13 and address two complementary questions about the selected interval.

The GAP Head of the Race Change lollipop chart answers one of the most important questions in stage racing: *Who gained time on the leader, and who lost it?* The x-axis is the time variation in seconds relative to the head of the race between the start and end instants. A vertical yellow line marks zero. Bars extending to the right are green and represent time gained; bars extending to the left are red and represent time lost. Riders are sorted from the greatest gainer at the top to the greatest loser at the bottom, making the hierarchy of performance immediately readable. Hovering over a dot reveals the rider's name, team, the exact time gained or lost, and the group transition: the macro-group the rider belonged to at the start instant and the one they belonged to at the end instant (for example, *Peloton* → *Chase Leader*). The macro-group computation is explained in the next section. This group context, shown inline in the tooltip, adds a layer of tactical interpretation that a plain gap figure cannot provide.

Placed alongside it, the Position Change slope chart addresses a complementary question: *How did riders move through the race in terms of position?* Each rider is a line connecting their position at the start instant (left) to their position at the end instant (right). The y-axis is inverted so that position 1 (the leader) sits at the top. Lines sloping upward represent riders who gained places; lines sloping downward represent riders who fell back. Each line is colored in the rider's team color. The slope chart was chosen

because it encodes direction of change through the angle of the line, which is perceptually more immediate than comparing two separate bar charts. When multiple lines of the same color all slope in the same direction, it reveals a coordinated team performance.



Figure 13: Gap Lollipop Chart and Position Slope Chart

Looking at the charts, our Movistar fan can get some more insights here. For example, *M.A.Lopez* was the rider who shortened the most his gap with respect to the leader and also gained more positions from Movistar team (by looking at the Slope Chart tooltip), going from the 47th position to the 24th position, closer to some of his teammates who were better classified in the first instant.

The lollipop chart is built with *Apache ECharts* using two overlaid series on the same instance: a bar series and a scatter series for the dot at the tip. The zero line is rendered via `markLine` on the bar series anchored at the x-axis 0 position, styled in yellow. The y-axis uses the same rich text label formatter as the heatmaps, coloring each rider name in their team's official color. On every filter change the chart instance is fully cleared and redrawn via `chart.clear()` followed by `setOption`, since the rider set and sort order both change.

The slope chart is built with *Plotly.js*. Each rider is a separate scatter trace in `lines+markers` mode, with two categorical x-axis values, *KM Start* and *KM End*, and the rider's race position at each endpoint as the y value.

Both charts are built from the same two rider snapshots: `lastData1Ref` and `lastData2Ref`, which hold the full per-rider records returned by `/api/instant?t` for the start and end instants respectively. When any of the interval changes, the page fetches the snapshot and stores it in these refs, and immediately draws the charts. When only the global filter changes, both charts are redrawn from the cached refs with no additional network request.

4.5.4 Group Transition Sankey

In order to display group transition between the two selected instants, no chart solved this better than The Sankey diagram (Figure 14). It shows how riders moved between race groups between the two selected instants. Unlike the other charts, the Sankey always displays all riders in the race regardless of the global filter, because its value lies in revealing the full picture of group dynamics rather than the behavior of a

specific subset.

A challenge encountered during development was that towards the end of a stage the raw number of distinct groups in the database grows substantially, as the peloton fragments into many small chase groups and straggler pockets. Displaying all of them would produce an unreadable diagram with dozens of thin bands. To address this, a set of macro-groups was defined to aggregate the raw groups into a small set of meaningful tactical categories, defined in Table 2.

Group	Description	Color
Front Group	Riders at the front of the race	White
Leader Group	The group containing the race leader	Yellow
Chase Leader Group	Chase groups directly pursuing the Leader Group	Amber
Chase Front Group	Chase groups directly pursuing the Front Group	Light green
Peloton	Main peloton	Light blue
Stragglers	Riders dropped from the peloton	Orange
Back	Riders at the very back of the race	Red

Table 2: Macro-Groups Description

The left column of the diagram represents the macro-groups at the start instant; the right column represents the macro-groups at the end instant. Nodes are colored by group and ordered from top to bottom by their position at the front of the race. The bands connecting left to right encode how many riders moved from one group to another: a thicker band means more riders made that transition. Band color is taken from the source group.

The classification of chase groups into *Chase Leader* vs. *Chase Front* is determined contextually: a chase group is labelled *Chase Leader* if the Leader Group is ahead of it, and *Chase Front* if the Front Group is the closest group ahead. This distinction separates riders chasing a GC relevant leader from riders chasing a breakaway that may carry no overall classification significance.

Clicking any band opens a popup modal view listing all the riders that made that specific transition, sorted by their end position. Each row shows the rider's start and end race position at both endpoints. The modal can be dismissed by clicking outside it or pressing Escape.

Movistar can analyze in Figure 14 how the initial peloton breaks completely into chase groups as the race breaks itself in the final stages and a number of straggler groups reorganize in a new peloton.

The Sankey is built with *Apache ECharts* using a `sankey` series with `orient: horizontal`. A key implementation decision is setting `layoutIterations: 0`, which disables the ECharts algorithm that normally reorders nodes to minimise band crossings. Without this, ECharts would re-sort the groups and break the top-to-bottom ordering by race position that makes the diagram tactically readable. Nodes are built as two independent sets, one per endpoint, each assigned `itemStyle.color` from the macro-group palette and a label anchored to the left or right edge of the chart. The side margins are set to 130 px on each side so that group label text is never clipped. The chart instance is fully cleared and rebuilt via `chart.clear()` and `setOption` on every interval change.

The click handler is registered once on the chart instance. It looks up the corresponding rider list in a `sankeyTransRef` map populated during the last call, keyed by the source–target macro-group pair, and passes the result to *React* state to render the popup overlay.



Figure 14: Group Sankey Diagram

Like the lollipop and slope, the Sankey is driven by `lastData1Ref` and `lastData2Ref`. However, it does not respect the global filter: `drawSankey` always receives the full rider snapshots, so every rider in the race contributes to the group flow regardless of the active selection. The macro-group assignment for each rider is computed entirely on the client side. No additional network request is made, since both snapshots are already fetched in the same parallel call as the lollipop and slope data.

4.6 Analytics

While the previous pages allow the user to explore the race moment by moment or compare two instants, the *Stage Analytics* page takes a different standpoint: it provides a whole-stage analysis. Rather than asking *What was happening at this instant?*, the Analytics page asks *What story does the entire stage tell?* All five charts on this page are built from precomputed static JSON files, since they aggregate information across the full instants of the stage and such operations would be too costly to run interactively against the database. The data preprocessing pipeline that produces these files is described in Section 3.2.

This page begins with the same global filter as Stage Interval, which allows selecting between the Top 20 finishers, up to 20 specific riders or up to three specific teams. This filter is explained in section 4.5.1, so the next sections will explain directly the charts.

4.6.1 Gap Evolution

Even though all charts in the previous pages solve very important problems, yet there is no chart that gives a full stage picture of the race evolution. For this reason, the first chart in this page is a GAP HEAD OF THE RACE line chart (Figure 15) of the entire stage, which allows to see from start to finish the evolution of each rider gap, which is a very visual indicator on how it was positioned in the race.

It is a *Plotly* line chart with the instant number on the x-axis and the gap in seconds on the y-axis. The y-axis is inverted: zero (the leader) sits at the top, and larger gaps appear lower, so a line dropping downward means the rider is falling further behind and a line climbing upward means they are closing on the front. Lines are colored by team, maintaining coherence with the other charts in the App, and the tooltip the exact rider's name.

In Teams mode, a sub-picker appears below the chart, allowing the user to toggle individual riders within the selected teams on and off, reducing clutter while keeping the team-level filter active for all other charts.



Figure 15: Gap Evolution Chart

Here, our Movistar fan can make further analysis on the development of the race, such as looking at how *I.Garcia Cortina* and *I.Erviti* were really close to *M.Mohoric* during a long part of the race and then they fell behind, finishing at near values of some of their Movistar teammates. It is also interesting to see how the other riders apart from these two stayed close to each other till near the end, where the race broke its main groups and riders began to separate, either closing the gap or falling apart.

The chart is drawn whenever the filter changes. The chart is built with *Plotly*: one line per selected rider, drawn as a smooth spline curve rather than a sharp angular path. Before each line is drawn, *cutAtFinish()* is applied to the rider's gap values: it scans from the end of the array, detects where the gap value stops changing, which marks the moment the rider crossed the finish line, and removes that flat tail so the line terminates cleanly at the finish rather than extending as a horizontal plateau. The y-axis is inverted so that a gap of zero seconds appears at the top, placing the race leader above everyone else and making it visually natural to read the chart as a ranking. On every subsequent filter change, *Plotly* updates the chart in place rather than rebuilding it from scratch, keeping the interaction responsive.

The chart is driven entirely by *rider_gap_evolution.json*. The file contains an *instants* array used as the shared x-axis, and a *riders* array where each entry holds *num*, *rider*, *team*, and a *gap* array of the same length as *instants*. The JSON is fetched once at page initialisation, together with the other five static files.

4.6.2 Rider Group Evolution Chart

The Rider Group Evolution chart (Figure 16) was thought to answer two questions that the Gap Evolution chart cannot: *Which groups was each rider part of along the race? When was the rider part of each group?*

It is a Plotly horizontal bar chart where each row represents one rider and the x-axis spans from the first to the last instant of the stage. Each row is divided into colored segments, where each segment represents a continuous run or period during which the rider remained in the same macro-group without interruption. The segment colors are identical to the macro-group palette used in the Sankey diagrams across the application, so the user can read the group membership history of any rider at a glance. Riders are sorted by descending final position of the race, so it can be rapidly seen how the best and worst performers evolved during the race.

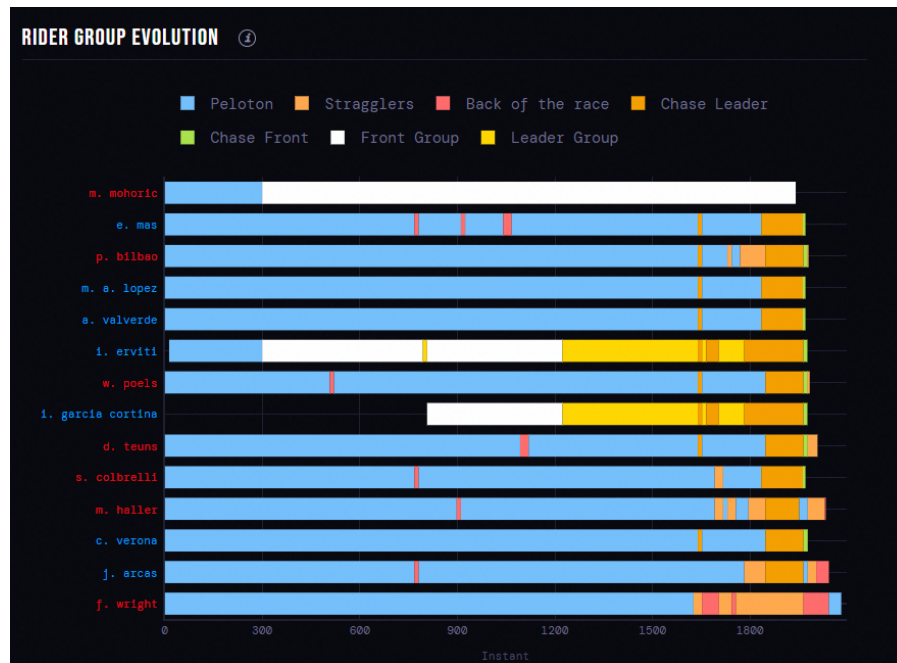


Figure 16: Rider Group Evolution Chart

The most notable things here is the confirmation that *I. Garcia Cortina* and *I. Erviti* were the best situated Movistar riders, which could not match the push of *M. Mohoric* towards the end of the race, where they fell from the Front Group. Also, the Movistar fan could also confirm here how *E. Mas* struggled during the race while being the best Movistar classified in the GC till this stage, falling to the back of the race at several points, but then recovering and putting himself into the chase groups.

The chart is built with Plotly as a horizontal bar chart where each bar represents a continuous period during which a rider remained in the same group. Every bar is anchored at its starting instant along the stage and extends to its ending instant, so together they tile each rider's row from left to right covering the full race. This is achieved through `barmode: 'overlay'`, which draws all bars at their exact precomputed positions rather than accumulating them, ensuring each segment sits precisely where it belongs on the stage timeline. The color of each bar is drawn from the macro-group palette, the same one used in the Sankey diagram, so the color meaning is consistent across pages. Bars belonging to the same group type share a single legend entry regardless of how many segments exist across all riders, keeping the legend compact. Rider names are not placed on the y-axis directly but instead added as floating labels on the left margin of each row, colored in their team's official color, so the team identity of each rider is immediately readable alongside their group history. Finally, the container height is computed dynamically based on the number of active riders, so each row maintains a legible size regardless of how many riders are displayed.

The chart reads `rider_race_shape.json`, which contains a `groups` array (each entry: `key`, `label`, `color`) and a `riders` array. Each rider entry holds identifying fields and `inst_s` and `inst_e` objects to situate each rider group in the race in terms of instants. The sort order is applied during the preprocessing step that generates the file, so no client-side sorting is needed.

4.6.3 Average vs Final Position

How can one identify which riders pushed harder at the end of the race? While this may sound like a straightforward question, it is far from trivial. In the closing stages of a multi-stage race, established group structures and team tactics frequently collapse as individual riders launch desperate, uncoordinated attacks. Capturing who truly excelled in these chaotic final moments requires an approach that isolates performance from standard, cumulative race variables. To address this, a dedicated visualization was conceived to provide an objective analysis free from the biasing influence of overall time gaps: the Average vs. Final Position Scatter Plot (Figure 17).

This chart places each rider at a point defined by two coordinates: their average on-road position across all instants of the stage on the x-axis, and their race position on the y-axis. Both axes are inverted so that position 1 (the best) sits at the top-left corner. A diagonal dotted reference line where $x = y$ is drawn across the chart, in order to know rapidly which riders finished better (above) or worse (below) its average position.

The diagonal line is the analytical core of this chart: a rider sitting on it had a final result consistent with their average position throughout the stage. Riders below the diagonal finished worse than their stage average, indicating they held a strong position for much of the race but deteriorated in the final kilometers. Riders above the diagonal finished better than their average, suggesting they made a decisive move late in the stage, a characteristic pattern for riders who save their effort for the final climbs. Each dot is colored in the rider's team color and hovering reveals name, team, final position and average position.

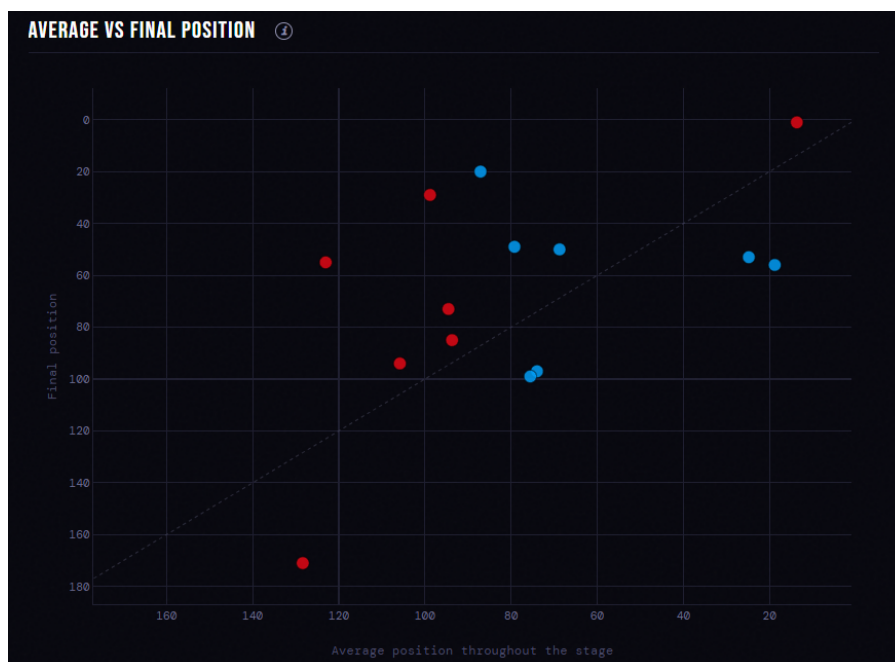


Figure 17: Average vs Final Position Chart

Here, it can be seen how *E.Mas* pushed hard in the final stages as its point is one of the most separated dots above the line, confirming what was seen in the previous chart. It is also notable how *I.Garcia Cortina* and *I.Erviti* are below the line, as they finished in a worse position than their average one. Lastly, it is also interesting to see the poor performance of *F.Wright* who fell off at the end but he was not performing that much better during the race.

This chart is builded using two *Plotly* traces. The first is a dotted diagonal reference line running from the best to the worst position, representing the point where average and final position are equal. The second trace is a mode: 'markers' scatter, where each marker represents one rider, sized and colored by their team. Both axes are inverted with `autorange: 'reversed'` so that position 1 appears at the top-left corner, making the chart read intuitively as a ranking rather than a raw numeric scale.

The chart is driven by `pos_avg_final.json`, which contains a `riders` array where each entry provides `num`, `rider`, `team`, `avg_pos` (mean on-road position across all instants) and `final_pos` (official finish classification).

4.6.4 Climber vs Sprinter

Cycling commentary routinely labels riders as "climbers" or "sprinters," but those labels come from reputation, not from the data of a specific stage. The following chart answers the question: *Given what actually happened in this stage, how did each rider perform on steep terrain versus flat terrain?* A rider might be a known sprinter but climbed well on this particular day or vice versa, so this chart lets the data speak instead of relying on pre-assigned archetypes.

The Climber vs Sprinter chart (Figure 18) classifies each rider by placing them at the intersection of two speed metrics:

- **x-axis:** average speed on road sections with gradient $\geq +5\%$
- **y-axis:** average speed on flat sections where $|\text{gradient}| < 5\%$

The 5% gradient threshold to classify a section as a climb is consistent with established conventions in cycling biomechanics: Martin et al. [14] demonstrate through force modelling that gravitational resistance becomes the dominant load in this range, and Padilla et al. [15] apply an equivalent criterion to separate flat and uphill performance in professional road cycling.

This axis choice directly captures the classical cycling archetype split: a pure climber is fast on steep gradients but relatively slow on flat terrain, while a pure sprinter excels on flat sections but loses ground on climbs. Two median crosshair lines (dashed, in yellow) divide the scatter into four quadrants and make this split explicit. The medians are computed from the full field of riders rather than from the active filter selection, so they remain stable reference lines as the user changes their selection as recomputing them on every filter change would make the quadrant boundaries shift and render comparisons meaningless. Corner annotations label the extremes: "*better climber*" in the bottom-right (high climb speed) and "*better sprinter*" in the top-left (high flat speed). Riders in the top-right quadrant are fast on both terrain types, the most complete all-round performers of the stage.

The axes are also fixed to the full-field range rather than to the active selection, for the same reason: zooming the axis to the visible subset would create a misleading impression that a filtered rider is average when they may in fact be an outlier in the full field.

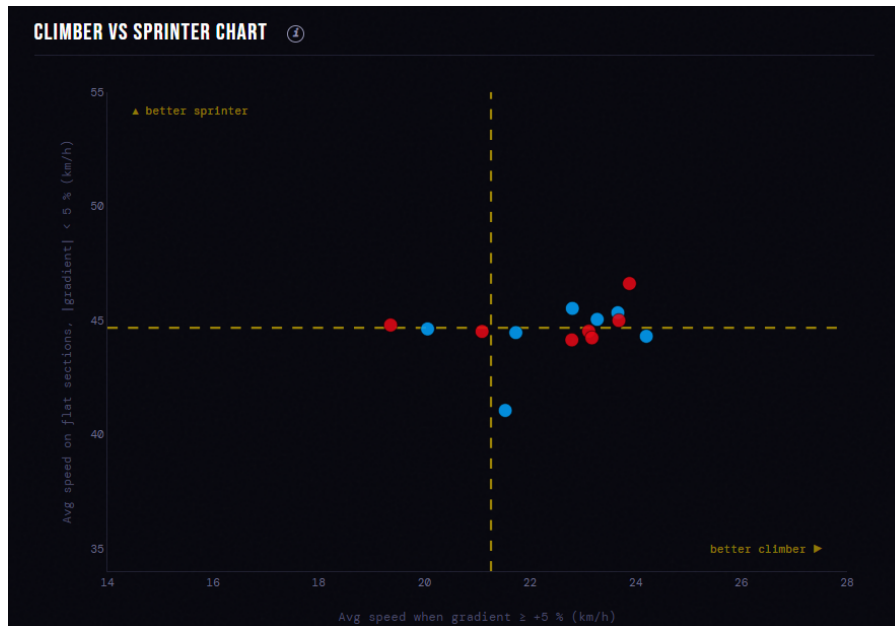


Figure 18: Climber vs Sprinter Chart

It is straightforward from Figure 18 how the majority of riders from the two studied teams are above the median in terms of climbing speed, being *C. Verona* the best with 24.2 km/h and *F. Wright* the worst with 19.4 km/h. Also, *S. Colbrelli* is the one that stands out globally as he is the best sprinter and the second best climber.

This chart is fully made with *Plotly*, using three traces. The first two are the median crosshairs, rendered as dashed yellow scatter traces spanning the full axis range. The third trace is the scatter of active riders, with mode: 'markers' and each dot colored by team. Both axes are set to a range fixed to the full-field extent and the background grid is hidden to avoid clutter with the median lines, being the tooltip the one that allows seeing the exact values. Corner labels are added via `layout.annotations` at specific data coordinates rather than as axis titles, preserving their position relative to the quadrant geometry.

The chart reads `climber_sprinter.json`, which provides a `riders` array where each entry holds `num`, `rider`, `team`, `climb_sp` (mean speed on gradient $\geq +5\%$ sections) and `flat_sp` (mean speed on sections where $|\text{gradient}| < 5\%$). The complete unfiltered array is retained in memory alongside the active subset so that the median computation always draws from the full field, independent of the current selection.

4.6.5 Rider Performance Radar

The last chart of the *Analytics* page allows the user to show and compare riders' performance across the race, summarized into 5 metrics. This is very important as was thought to answer the famous question of *Who performed better in this stage?* directly without much exploration. The Rider Performance Radar allows to overlay up to five selected riders on a single polygon chart where each axis represents a distinct performance metric:

- **Avg speed:** overall average speed across the entire stage (km/h).

- **Climb speed:** average speed on road sections with gradient $\geq +5\%$ (km/h).
- **Flat speed:** average speed on sections where $|\text{gradient}| < 5\%$ (km/h).
- **Descent speed:** average speed on sections with gradient $\leq -5\%$ (km/h).
- **Avg gap to leader:** On this axis rank 1 (outer edge) belongs to the rider who stayed closest to the front throughout the race.

The key design choice is that the axes encode field rank, not raw values. The centre of each axis corresponds to last place in the full field, and the outer edge corresponds to first place. A rider occupying the entire outer perimeter would rank first in every metric simultaneously. This rank-based encoding makes polygons directly comparable across metrics that have very different units and magnitudes, making straightforward who was better in that metric.

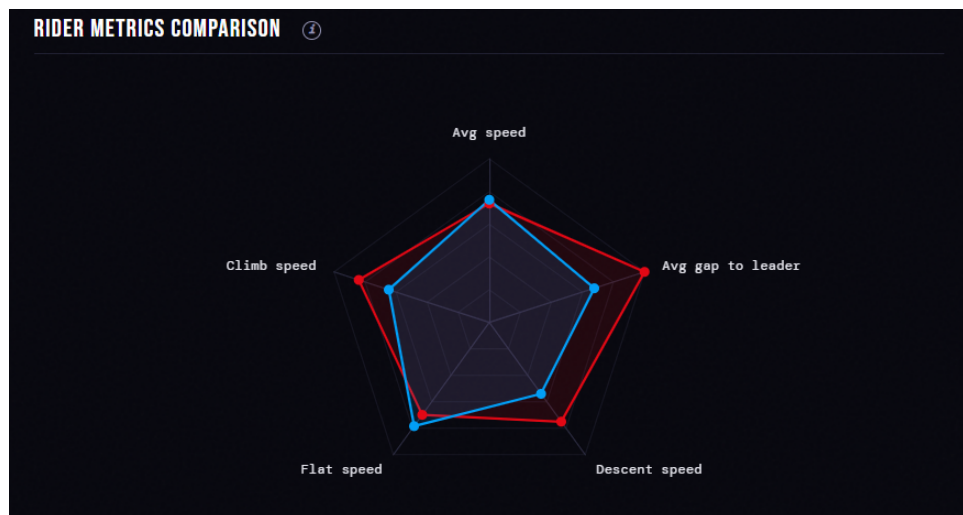


Figure 19: Rider Performance Radar

Finally, here our Movistar fan can compare its team riders performance with whoever he wants. The case from Figure 19 shows the performance of *M. Mohoric* and *E. Mas*, two riders who have been talked a lot during all the analysis. It can be seen how *M. Mohoric* was better in the climbs and descents, while *E. Mas* was faster in the sprints. It is also interesting to see how the first rider was not in the Top-10 of any metric (apart from being closer to the leader, as he remained all the race in the front) and still managed to win the race thanks to its overall performance.

A picker below the chart lists all riders in the current active selection and allows the user to toggle up to five simultaneously. Whenever the team or rider filter changes, the displayed polygons are automatically trimmed to the first five riders from the updated selection so the chart always remains readable.

This chart is built with *Apache ECharts* using the radar series type with a polygon shape and five named indicators. Each axis value is a normalised score between 0 and 1, where rank 1 in the full field reaches the outer edge and last place collapses to the centre. One series is created per displayed rider with a low fill opacity, so overlapping polygons remain distinguishable. On hover, the tooltip iterates the five

metrics and for each one shows the rider's raw value alongside their field rank (e.g. *Avg gap to leader: 324 s #12/177*).

The chart reads `rider_radar.json`, which contains a `metrics` array (key, label, and unit for each axis) and a `riders` array. Each rider entry holds their raw metric values, a precomputed `rank` per metric, and a precomputed `rank_value`, which is the normalised score ready to pass directly to ECharts. Ranks are computed during preprocessing for all metrics at once: speed metrics are ranked descending (higher is better) and average gap ascending (lower is better). Each rank r is then converted to an axis value using

$$\text{rank_value}(r) = \frac{N - r}{N - 1}$$

where N is the total number of riders. This maps rank 1 exactly to 1.0 (outer edge) and rank N exactly to 0.0 (centre). Because the rank-to-value mapping is already resolved in the JSON, no sorting or computation is performed at draw time and the file is passed directly to the chart on every filter change.

5. App Evaluation

To evaluate the application, we are going to revisit the initial research questions, in order to see whether the application manages to answer them and what limitations it presents.

Q1: Behavior and evolution of a rider, team, or group throughout a stage

BQ3 *What are the time gaps between riders at a specific instant of the stage?*

Fully supported by the *Stage Instant* view. The user drags the instant slider to any point in the race; the ranking table updates in real time showing each rider's gap to the current race leader.

BQ6 *Which cyclists belong to each team?*

Supported across all views. Every ranking table and every chart displaying individual riders shows team affiliation alongside the rider's name, with a consistent color coding by team applied across all views.

SQ3 *Have there been attacks or breakaways during the stage?*

Supported by the *Analytics* page (Gap Evolution chart) and the *Stage Interval* view (Sankey diagram). In the Gap Evolution chart, breakaways are visible as lines that separate from the peloton and converge towards the lowest gap values. In the Sankey, it can be seen how groups change and whether any chase group formed from the peloton.

SQ4 *What groups formed during the stage? Do they represent a threat to the race leader?*

Addressed at different levels of detail across the pages. The *Stage Instant* view shows all groups and their members at any selected moment. Moreover, the stage ranking shows the group and the gap for each rider, which tells if a group is a threat or not. On the other hand, the *Stage Interval* view shows group transitions between two instants, which answers the first question, and by clicking the transition it can be seen each instant riders gaps, which answers the second.

SQ5 *What is the time gap between a contender and the leader at a specific point?*

Addressed jointly by the *Stage Instant* view and the *Gap Evolution* chart. The instant view gives a precise point-in-time gap for any rider; the Gap Evolution chart shows how that separation evolved across the full course of the race.

AQ1 *How did a rider's position evolve over a given interval?*

Fully supported by the *Stage Interval* view through the Position Change (Slope) chart.

AQ3 *What was the likely strategy of a team at a specific moment?*

Partially supported through the combination of *Stage Instant*, *Stage Interval*, and *Analytics*. The first shows the team's situation at a given point; the second shows group changes over a selected passage; and the third gives a stage-wide view that allows hypotheses to be confirmed or refuted. This question cannot be answered at a glance, but the application provides the tools to address it in detail.

AQ6 *How did a team evolved and behave across the stage?*

Team filtering is available in *Stage Instant*, *Stage Interval* and *Analytics*. This allows to see the evolution of its team riders GAP HEAD OF THE RACE and other metrics at any instant/interval selected. Moreover, *Analytics* shows whole-stage aggregated charts to further inspect team behaviour and how its riders performed at interested moments. Therefore, this question can also be considered fully supported.

Q2: General and stage rankings before, during, and after the stage

BQ1 *Who won the stage?*

Answered immediately by the *Stage Overview* page. The stage hero card prominently displays the stage winner and their team as one of its primary data fields.

BQ2 *What was the final finishing order?*

Answered by the Stage Ranking panel on the *Stage Overview* page.

BQ4 *Who leads the overall race after this stage?*

Answered by the General Ranking panel on the *Stage Overview* page. The top three positions are shown immediately.

SQ1 *Who is the virtual race leader at a specific point along the route?*

Supported by the *Stage Instant* view. The ranking at any selected instant reflects the cumulative general classification standings at that moment: the rider at position 1 with a gap of zero seconds is the virtual race leader.

AQ2 *Which riders gained or lost the most time relative to the race leader over a given interval?*

Fully supported by the *Stage Interval* view via the Lollipop chart. Once start and end instants are defined, the chart ranks all active riders by their net time gain or loss over the interval, displayed as horizontal stems extending right (gain) or left (loss).

AQ4 *What do the various classifications look like at a specific moment of the stage, and how do they evolve throughout?*

Partially supported. The *Stage Instant* view provides the live on-road main ranking at any selected instant. However, there is not a full aggregated chart showing the evolution of the ranking, so it can only be seen via exploration. Moreover, the exploration and evolution of secondary classifications — points, mountains, youth — during the stage is not surfaced: these change at intermediate sprints and summit passages and would require additional dedicated data fields and views. The *Stage Overview* page provides the full final classifications after the stage, but their intermediate evolution is an acknowledged gap in the current implementation.

Q3: Team performance and strategy in a specific stage

The *Analytics* page is the primary instrument for addressing this question. With the global filter set to Teams mode and up to three teams selected, all five charts update in concert to produce a coherent, multi-dimensional comparative portrait.

The Gap Evolution chart reveals which teams stayed together and which fragmented, and at which point of the stage any decisive separation occurred. The Rider Group Evolution chart complements this with a stage-wide structural view, showing the fraction of the race each rider spent in each group, making tactical patterns immediately visible across teams.

The Climber versus Sprinter scatter chart provides the most quantitative dimension of the comparison, plotting each rider's average speed on steep terrain against their speed on flat sections. This allows the user to assess whether a team's strength lies on the climbs, on the flat, or both. Finally, the Radar chart enables a direct multi-metric comparison of individual riders across all five performance dimensions simultaneously. Team filtering is also available in *Stage Instant* and *Stage Interval* for inspection at a finer granularity, adding high-level of detail to support this question.

Q4: Evolution of teams, riders, and groups at key moments of the stage

This question is addressed by the *Stage Interval* view, which is purpose-built for the analysis of bounded sections of the race. The user can isolate any key passage — a categorised climb, the breakaway formation, the final kilometers — by setting the start and end instant sliders to the approximate boundaries of that segment.

With these instants in place, three charts work in combination. The Lollipop chart immediately identifies which riders gained time over the interval and which lost it. The Slope chart shows position changes over the same interval for every active rider. The Sankey diagram reveals the group structure transition. Additionally, the Speed Heatmap provides a visual picture of the speeds of each rider in the selected interval, showing its evolution in the key point of the stage.

For someone who does not know what the key moment are, *Stage Overview* shows the principal climbs and sprints in order to guide their filtering in the page.

Q5: Stage characteristics and profile

Fully addressed by the *Stage Overview* page. The stage hero card presents the key facts of the stage in structured form: total distance, stage type, number and category of the climbs, and the location of the intermediate sprint. The elevation profile chart, drawn directly from the raw GPX data of the route, gives an immediate visual sense of the terrain: the location and relative steepness of each climb, the flat sections between them, and the character of the final kilometers. Together, these two elements ensure that any user, even one unfamiliar with the stage, can rapidly grasp its profile before engaging with the deeper analytical views.

Then, in the *Stage Instant* and *Stage Interval* pages, this profile is used in the filtering as well, in order to do a better contextualization of the selected instant/interval and allow richer analysis and exploration.

6. Conclusions

The evaluation carried out in Section 5 allows us to revisit the four goals set out in Section 2 and assess the degree to which the application fulfills each of them.

Deep exploration and broad analysis. The application was designed around a deliberate spectrum that ranges from exploratory to analytical. The *Stage Overview* page serves as an entry point, providing the essential context of the stage without requiring any interaction. The *Stage Instant* view is fully exploratory in nature: it queries the database at any instant the user selects, surfacing fine-grained information at the highest level of temporal detail the data supports. The *Analytics* page occupies the opposite end: all its charts are built from precomputed, race-wide aggregations, enabling comparative and summary analysis across the full stage without any real-time query overhead. The *Stage Interval* page sits between the two, combining live database queries at two selected instants with precomputed speed data, making it both exploratory and analytical depending on how the user engages with it. This spectrum, from instant-level exploration to stage-wide analysis, was one of the central architectural decisions of the project, and the evaluation confirmed that each view fulfils its intended role. This goal is considered fully achieved.

Exploitation of high-granularity data. The Stage Instant and Stage Interval views are the direct expression of this goal. By querying per-rider telemetry at every recorded instant of the race, they expose a level of detail that no tool in the state of the art can provide. The evaluation demonstrated that questions requiring this granularity — such as AQ1 (position evolution over an interval), AQ2 (time gained or lost over a given passage), AQ3 (team strategy at a specific moment), AQ6 (team behavior across the full stage), Q3 (team performance and strategy) or Q4 (evolution at key moments) — are all addressed by the application effectively. The exploitation of high-granularity data is therefore the single most significant contribution of this project relative to the state of the art, and this goal is considered fully achieved.

Balance between clarity and information density. Considerable development effort was invested in the design of each view: color coding by team applied consistently across all pages, contextual InfoTip tooltips on every chart, adaptive filtering systems that allow the user to restrict each visualization to precisely the riders or teams of interest, and a typographic and color palette designed to remain readable at high information density without becoming visually overwhelming. Every chart has an explicit purpose and a self-contained analytical role within its page. The evaluation confirmed that all views are functionally usable and that the information is conveyed effectively. It is honest to acknowledge, however, that the time devoted to interface design decisions was time not spent building additional visualizations, a trade-off that was conscious and deliberate but meant less focus on additional visualizations.

Taken together, all three goals have been substantially achieved in this first implementation. The evaluation in Section 5 showed that the application answers all Basic and Specific Questions completely, and that the Analytical Questions — particularly those requiring per-rider, per-instant telemetry — are addressed at a depth that no existing tool provides.

7. Future Work

The application is functional but in its first stages of development. The following directions represent the most relevant next steps.

- **User evaluation.** The application has not yet been formally evaluated with real users. A structured study with cycling fans and analysts — collecting usability feedback, task-completion data, and qualitative impressions — would validate the design decisions and identify the most impactful areas for improvement.
- **Deployment to production.** Future work will focus on transitioning the application from a local setup to a continuous production environment. This will involve containerizing the backend architecture to ensure consistent deployment, migrating the database to a managed cloud service for scalable data storage, and configuring a dedicated web server to host both the APIs and the application continuously. Together, these upgrades will eliminate the need for local execution and make the entire platform permanently accessible over the web.
- **Performance and scalability.** The current schema works well for a single stage and is built with scalability in mind. However, it needs further optimisation to handle multiple stages and races effectively with minimal human intervention.
- **Expanding to more stages and competitions.** Only one stage is currently implemented. A natural next step is to incorporate all stages of the 2021 Tour de France using the existing data pipeline, and subsequently extend to other Grand Tours and one-day Monuments such as the Giro d'Italia or Paris–Roubaix.
- **Accessibility.** Color is the primary visual encoding across all charts, with no fallback for color-vision deficiencies. Future work should introduce alternative encodings and keyboard navigation to meet WCAG 2.1 AA standards [16].
- **Real-time data pipeline.** Connecting the backend to the live GPS feeds produced during broadcasts would allow the application to act as a live analytical companion, updating in synchrony with the race.

References

- [1] Amaury Sport Organisation (A.S.O.), “Tour de France by ŠKODA”, Mobile application, Apple App Store.
- [2] ProCyclingStats, Cycling statistics, results and rankings[Online]. Available: <https://www.procyclingstats.com/>
- [3] FirstCycling, “FirstCycling – Cycling statistics, results and rankings”. [Online]. Available: <https://firstcycling.com/>
- [4] ProTour Cycling, “ProTour Cycling – Cycling results and statistics database”. [Online]. Available: <https://www.protourcycling.com/>
- [5] CyclingArchives, “Cycling Archives – historical cycling database and results”. [Online]. Available: <https://www.cyclingarchives.com/>
- [6] CQ Ranking, “CQ Ranking – Cycling statistics and results database”. [Online]. Available: <https://cqranking.com/>
- [7] Cyclingoo, “Cyclingoo – Cycling live tracking and statistics mobile application,” Mobile application. [Online]. Available: <https://www.cyclingoo.com/>
- [8] Tour Tracker, “Tour Tracker Pro Cycling – Live race tracking application,” Mobile application. [Online]. Available: <https://www.tourtracker.com/>
- [9] J. Díaz, M. Fort, and P. Vázquez, “TourVis: Narrative Visualization of Multi-Stage Bicycle Races,” EuroVis 2021.
- [10] Tour de France, “Official website of Tour de France 2026.” [Online]. Available: <https://www.letour.fr/en/>
- [11] La Flamme Rouge, “La Flamme Rouge – Tous les jours, la passion du cyclisme.” [Online]. Available: <https://laflammerouge.com/>
- [12] C. A. Brewer, “ColorBrewer 2.0,” Pennsylvania State University, State College, PA, USA, 2002. [Online]. Available: <http://colorbrewer2.org/>.
- [13] Escola Tècnica Superior d’Enginyeria de Telecomunicació de Barcelona (ETSETB), “Guia de sostenibilitat per a Treballs de Fi de Grau i de Màster,” Universitat Politècnica de Catalunya (UPC), Barcelona, Spain. [Online]. Available: <https://www.etsetb.upc.edu/>
- [14] J. C. Martin, D. L. Milliken, J. E. Cobb, K. L. McFadden, and A. R. Coggan, “Validation of a mathematical model for road cycling power,” *Journal of Applied Biomechanics*, vol. 14, no. 3, pp. 276–291, 1998.
- [15] S. Padilla, I. Mujika, G. Cuesta, and J. J. Goiriena, “Level ground and uphill cycling ability in professional road cycling,” *Medicine & Science in Sports & Exercise*, vol. 31, no. 6, pp. 878–885, 1999.
- [16] W3C, “Success Criterion 1.4.1 Use of Color,” in Web Content Accessibility Guidelines (WCAG) 2.1, 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/#use-of-color>

A. Sustainability Analysis and Ethical Implications

A.1 Sustainability Matrix

Table 3 summarises the sustainability analysis of this project following the framework proposed by the ETSETB guide [13].

	TFG Development	Project Execution	Risks & Limitations
Environmental	Low footprint: ~ 350 h of development on a personal laptop (≈ 0.045 kW) yields ≈ 3.5 kg CO ₂ at Spain's average grid intensity of 0.22 kg CO ₂ /kWh. No physical materials were produced or consumed.	The application runs locally on a personal computer alongside a local PostgreSQL database. No dedicated server infrastructure is required, so the operational footprint is negligible beyond ordinary laptop usage.	If the project were scaled to a public multi-user deployment, server and database infrastructure would be needed, increasing energy consumption proportionally.
Economic	~ 350 h of development at a junior engineer rate of 15€/h $\approx 5,250$ € in human cost. Only paid tool: L ^A T _E X licence (≈ 50 € over 5 months). All other tools (Python, PostgreSQL, React, FastAPI) are free and open-source.	Case 1 (mini-server): hardware ≈ 200 € one-time + 9€/year electricity. Case 2 (cloud): ≈ 30 –50€/month for a small VM + managed database.	Changes to the source website could break data collection and require rework. Currently non-commercial: the scraped data is sourced without a formal licence agreement, which precludes any commercial use or public deployment without prior authorisation from the data provider.
Social	Significant personal learning in full-stack engineering, data pipelines, and visual analytics. The project required engagement with real professional sport data, raising awareness of how analytical tools are used in elite cycling.	Democratises access to professional-grade race analysis, previously available only to team analysts and broadcasters. Direct beneficiaries: cycling fans, journalists, coaches, and amateur analysts. Potential barrier: the interface complexity may exclude casual or beginner users.	The data covers only men's professional cycling, which may inadvertently reinforce the existing visibility gap between men's and women's cycling. Color-based encoding without fallbacks currently presents accessibility barriers for users with color vision deficiencies.

Table 3: Sustainability matrix of the presented project

A.2 Ethical Implications

The race data used in this project was scraped from publicly available sources and is employed exclusively for educational and non-commercial purposes. The riders' names, positions, and times displayed are information that was broadcast publicly during the event and present no individual privacy concern beyond what is already in the public domain. No personal data beyond sporting performance is stored or displayed.

The application presents all teams and riders on equal footing: no team is highlighted or hidden by design, and any data gaps in the database result from the original data collection process and are distributed randomly across riders and teams, introducing no systematic bias. The work was carried out following the

UPC Code of Ethics, and the use of open-source technologies throughout ensures that the methodological choices are transparent and reproducible.

A.3 Relation to the Sustainable Development Goals

The project connects, to varying degrees, to the following UN Sustainable Development Goals (SDGs):

- **SDG 3 — Good Health and Well-being:** by promoting engagement with professional cycling, the application contributes indirectly to a culture that values sport and physical activity.
- **SDG 9 — Industry, Innovation and Infrastructure:** the project demonstrates the application of open-source data engineering and interactive visualization to a real-world analytical problem, contributing to the broader ecosystem of open innovation in sports analytics.
- **SDG 10 — Reduced Inequalities:** professional race analysis tools are currently accessible only to well-funded teams and broadcasters. Making equivalent analytical depth available openly reduces the information asymmetry between professionals and the general public.
- **SDG 17 — Partnerships for the Goals:** the entire technology stack (Python, PostgreSQL, FastAPI, React, Plotly, ECharts) is open-source, reflecting the principle of knowledge-sharing and collaborative development that underpins SDG 17.

A.4 Use of Artificial Intelligence

Generative AI tools were used in a supporting role at different stages of this project. *Claude* (Anthropic) assisted primarily with $\text{\LaTeX}$ typesetting — checking syntax, formatting tables and figures, and improving the redaction of written sections — as well as with specific frontend questions, such as chart configuration and component structure in React.

Gemini (Google) was used during the research phase to explore related work, identify relevant libraries, and gather background on the state of the art in sports analytics and data visualization.

In both cases, the AI was used as a consultation aid. All analytical decisions, design choices, and implementations were made and verified by the author.

B. Preprocessing Charts

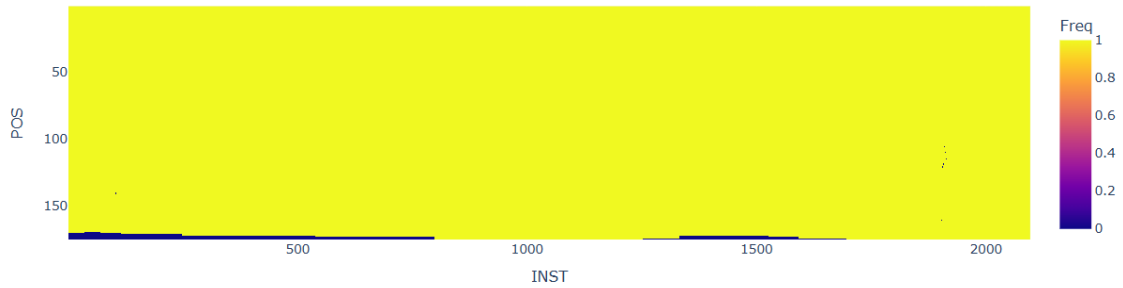


Figure 20: Position Values Chart

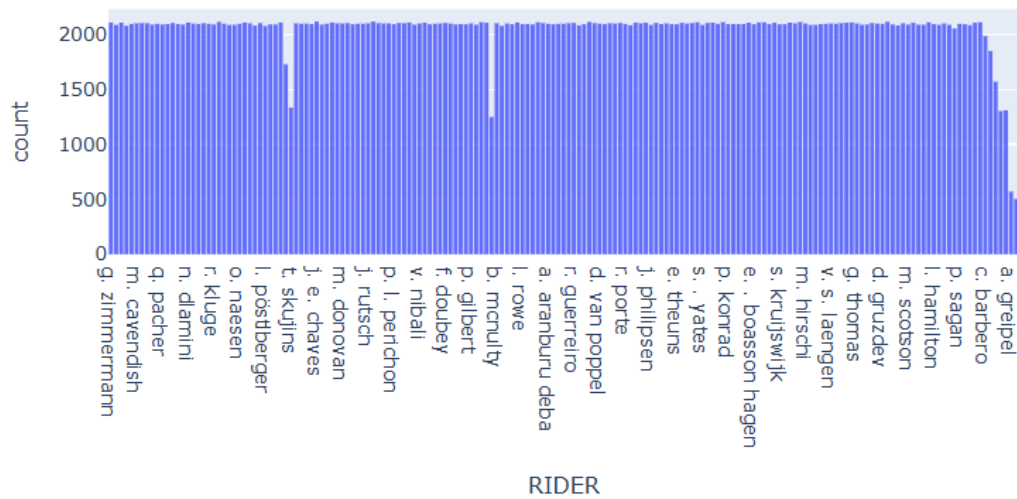


Figure 21: Rider Values Bar Chart

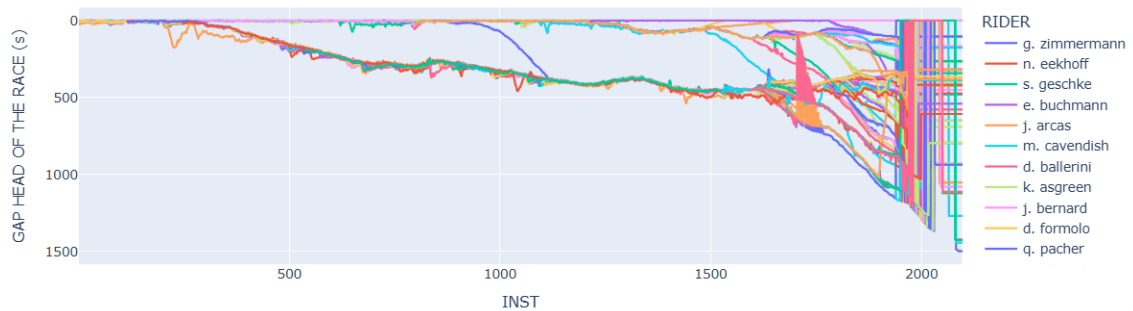


Figure 22: Uncleaned GAP HEAD OF THE RACE (s) Line Chart

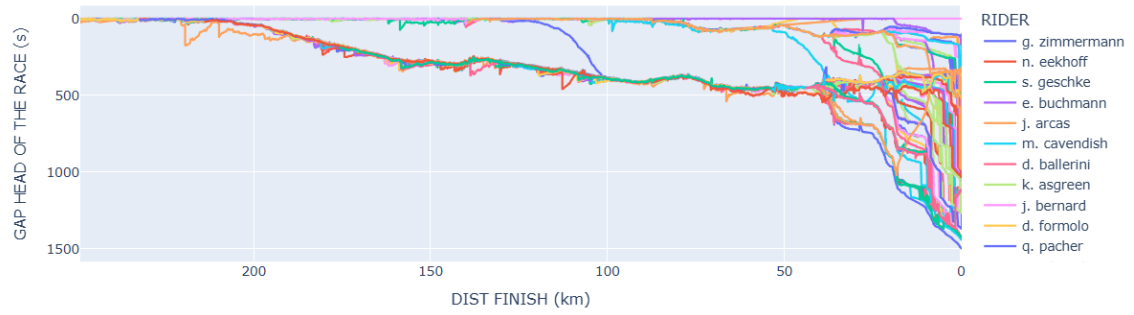


Figure 23: Cleaned GAP HEAD OF THE RACE (s) Line Chart

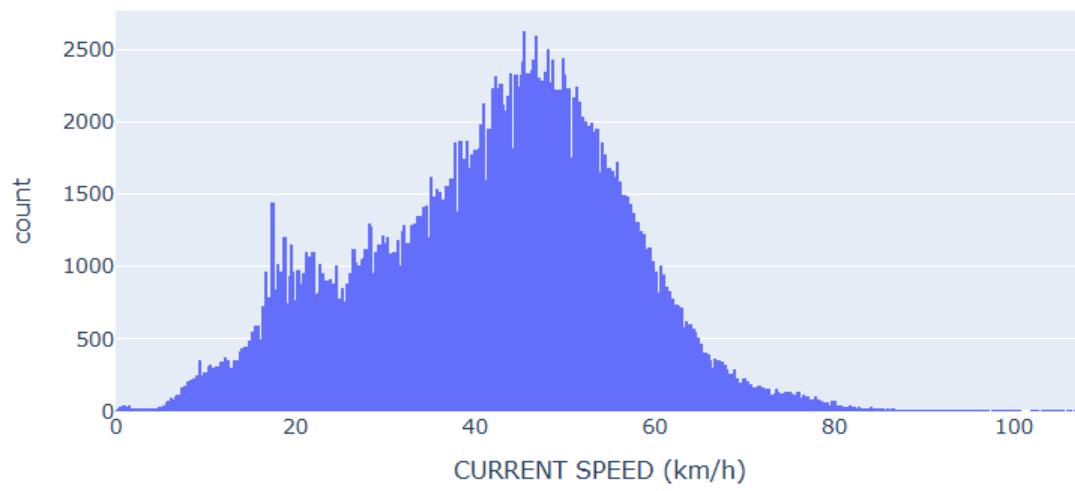


Figure 24: Speed Values Distribution Chart